Distributed Systems - Notes - v0.2.0

260236

October 2026

Preface

Every theory section in these notes has been taken from the sources:

- Course slides. [1]

About:

GitHub repository

These notes are an unofficial resource and shouldn't replace the course material or any other book on distributed systems. It is not made for commercial purposes. I've made the following notes to help me improve my knowledge and maybe it can be helpful for everyone.

As I have highlighted, a student should choose the teacher's material or a book on the topic. These notes can only be a helpful material.

Contents

1 Introduction **4**
1.1 Why Distributed Systems? 4
1.1.1 Motivation: Why Distributed Systems? 4
1.1.2 Why Distribute a System? 14
1.1.3 Why Distributed Systems Are Special 18
1.2 What is a Distributed System? 23
1.3 Middleware 28
1.4 Fundamental Properties 31
1.5 Main Challenges 34
1.6 Transparency 46

2 Modeling **52**
2.1 Software Architecture and Middleware 53
2.2 Client-Server and Multi-Tier Architectures 62
2.3 Service-Oriented Architecture (SOA) 72

Index **86**

1 Introduction

1.1 Why Distributed Systems?

1.1.1 Motivation: Why Distributed Systems?

There is a very simple reason why distributed systems are important: most applications we use today are **distributed**. Many applications cannot realistically run as a single program on a single computer.

> **Example 1: Distributed Applications**
>
> *E-mail*, web/cloud applications such as *Google Maps*, *Google Docs*, *Microsoft Office 365*; mobile applications and mobile devices, web services and websites, and many more, are all distributed systems.
>
> E-mail is used as an especially clear example because sending a message involves several independent machines communicating over a network:
>
> User A → mail server A → network → mail server B → User B
>
> There is no single computer performing the entire operation.

So the first intuition is that modern applications are naturally composed of multiple communicating machines. This is why distributed systems are not some niche branch of computer science, they are behind a huge fraction of modern computing infrastructure.

State of the Internet

Censys on 2023 created a report about the **observable public Internet**, built from large-scale Internet scanning.[1] The report says that in one daily snapshot in early 2023, Censys observed more than **740 million hosts** exposing about **1.3 billion HTTP services**, roughly **18%** hosted by major cloud providers such as Amazon, Oracle, Google Cloud, or Azure. And more than **85%** of HTTP services running on named hosts.

A distributed system may therefore consist not of two or three machines, but potentially of 10^3, 10^5, 10^6, or even more interacting components. But this immediately hints at problems that will appear later in the notes: scalability, failures, communication, naming, synchronization, replication, and so on. But at this point, we are just trying to understand why distributed systems are important.

Large-scale Web traffic

From the same Censys report, we can also see the **traffic** perspective rather than the number of machines. For example, the report says that in 2023, on Google:

[1] Censys is a platform that **scans and indexes Internet-connected systems and services**. It is like a search engine for Internet infrastructure.

- More than **100k searches per second**
- More than **100 billion visits per month**
- YouTube as another extremely large service
- Google and YouTube producing more traffic than many other major websites combined

Again, the exact values are secondary. The important question is: ***could a single server handle all of this?*** Clearly not. A large Web service therefore has to distribute work across many machines. And this is the first glimpse of the **scalability** problem that distributed systems have to solve.

In summary, modern computing operates at a scale where distribution is not just a design choice, but unavoidable. We saw how widespread distributed applications are, the size of the Internet, and finally the traffic generated by huge Web services.

LLM infrastructures as distributed systems

Let's consider what happens when we send a query to a modern Large Language Model (LLM) such as ChatGPT. It is tempting to imagine:

$$\text{Our request} \rightarrow \text{one very powerful GPU} \rightarrow \text{our answer}$$

But this is not how it works. Modern LLMs are too large to fit on a single GPU. So a single query to a modern LLM may involve roughly **16 to 32 GPUs**, corresponding to a couple of multi-GPU nodes. Each GPU may consume up to about **700 W**.

So even a single logical request may require computation spread across several physical processors:

$$\text{Request} \rightarrow \begin{cases} \text{GPU}_1 \\ \text{GPU}_2 \\ \text{GPU}_3 \\ \vdots \\ \text{GPU}_{16-32} \end{cases} \rightarrow \text{Answer}$$

From the user's perspective, however, we do not see any of this. We simply interact with one service: LLM service. This is already one of the central ideas of distributed systems: many independent computational resources cooperate to provide what appears to the user as one single service. The infrastructure becomes even more clearly distributed when we move from one request to the whole data center. An AI data center may contain **more than 100,000 AI accelerators**, with power consumption potentially reaching hundreds of megawatts.

So there are actually **two different forms of distribution** here:

- At the level of a **single request**:

$$\text{One Computation} \rightarrow \text{Multiple GPUs}$$

- At the level of the **service as a whole**:

$$\text{Millions of requests} \rightarrow \text{Many machines} \rightarrow \text{Many clusters} \rightarrow \text{Possibly many data centers}$$

The reason this example matters is that distribution is not only about connecting independent applications. Sometimes even one computation is too large to execute efficiently on one processor. That naturally connects distributed systems with high-performance computing (HPC), although the two are not identical. However, modern AI services are distributed both because of computational size and service scale.

Google as a distributed "supercomputer"

Eric Schmidt (CEO of Google from 2001 to 2011) described Google as essentially "*a large distributed supercomputer spread across many data centers*". That is a very useful mental model. Instead of imaging Google as a single website, we can think of it as an enormous collection of cooperating computers spread geographically across the world.

$$\text{Data center A} \leftrightarrow \text{Data center B} \leftrightarrow \text{Data center C} \leftrightarrow \dots$$

And inside each data center, there are many cooperating machines:

$$\text{server}_1,\ \text{server}_2,\ \text{server}_3,\ \dots$$

All connected through networking infrastructure.

Google historically operated hundreds of thousands of servers and gives a more recent estimate of **more than 2 million servers across 36 data centers**. The important point is not the exact count; it is the architectural idea. From outside, we might type `google.com` and perceive a single service. But internally, that service may involve an enormous distributed infrastructure. So we get an important contrast:

- **User's view**: Google, a single website.
- **Physical reality**: millions of machines, networks, and data centers cooperating to provide a single service.

This is a recurring objective in distributed systems: expose a coherent service while hiding much of the underlying distribution. We will later give this idea a precise name: **transparency**.

⚠ There is also another important implication. If the system contains millions of machines, failures cannot be treated as exceptional events. Suppose each individual machine is highly reliable. With millions of components, some machine

somewhere will still fail regularly. So at very large scale, **failure** is not something that might happen once in a while. It becomes something the system must be designed to live with. This point will become central when we discuss **partial failures** and **fault tolerance** later. For now, the main takeaway is that a huge distributed infrastructure can behave externally like one computer/service.

Distributed systems beyond the Internet

At this point it would be easy to conclude that distributed systems are only internet applications. That would be **wrong**. The Internet, the Web, and LLM platforms are distributed systems, but many distributed systems have little or nothing to do with the public Internet.

Example 2: Distributed Systems Beyond the Internet

- **Intranets**. An organization can have a private distributed system. For example, banking systems, corporate systems, scientific computing infrastructure, server farms, grids, clouds.

  For example, imagine a bank with branches in Milan, Rome, and Turin. The system is distributed even if its internal communication is largely private and not exposed to the public Internet.

- **Home networks**. Even our home can contain a small distributed environment: laptop, router, tv, NAS, phone, and so on. Different devices communicate and share resources. This is a distributed system on a dramatically smaller scale than Google, but the idea is the same: independent devices and communication.

- **Personal Area Network (PAN)**. A person's d devices can themselves form a small network: phone, smartwatch, headphones, gps. For example healthcare-monitoring devices can communicate with a phone and send data to a cloud service. Again, distribution does not imply thousands of servers. Two or three cooperating devices are already enough to introduce distributed-system concepts.

- **Wireless Sensor Networks**. Another important category consists of many small sensing devices communicating with one another or with gateways. For example, a smart home may have many sensors and actuators communicating with a central hub. Or a smart city may have thousands of sensors monitoring traffic, pollution, or infrastructure health. These are distributed systems even if they are not part of the public Internet.

Thus a distributed system may exist across continents, inside one company, inside one building, inside one house, or even around one person. What matters is not the geographical size, but that multiple independent computational components communicate and cooperate.

Home and personal networks

Until now, we have mostly thought about distribution at a very large scale: Google, cloud infrastructures, LLM data centers. But a distributed system can also exist on a much smaller scale.

For example, inside a home we might conceptually have a PC, a laptop, a media device, connected through a local network and possibly communicating with remote machines through the Internet. So distribution is not defined by physical distance. Two computers sitting in the same room can form part of a distributed system just as computers located on different continents can.

In the previous page, we mentioned home networks and Personal Area Networks (PANs). A home network concerns devices belonging to the environment:

$$\text{home} \rightarrow \{\text{computers}, \text{entertainment devices}, \text{shared resources}\}$$

A personal-area network concerns devices surrounding a person:

$$\text{person} \rightarrow \{\text{phone}, \text{smartwatch}, \text{headphones}, \text{gps}, \text{etc.}\}$$

⚠ The scale is small, but the distributed-system issue remains: **independent components must communicate and coordinate**.

Pervasiveness of Distributed Systems

Instead of having a few clearly recognizable computers (desktop, laptop, server), computing and communication capabilities become embedded in ordinary physical objects. This is the basic idea behind the **Internet of Things (IoT)**. We can use three quotations to develop this idea:

1. **Computing that "disappears"**. Mark Weiser (1991) says that the most profound **technologies** eventually become **integrated into everyday life** to the point that we **stop consciously noticing them**. This is the idea of **Ubiquitous Computing** (or **Pervasive Computing**).

   Think about the conceptual transition from "*use a computer*" toward "*computing is embedded in the environment around us*". The computer itself is no longer necessarily the object we interact with.

2. **Connectivity for things**. The ITU (International Telecommunication Union) quotation captures another transition. Traditional networking gave us connectivity from anyone to anywhere (i.e., from one person to another). IoT extends this toward **connectivity for anything**. Objects that previously had no communication capabilities can contain small transceivers and participate in a network. So instead of only connecting computers, we may have a network of connected objects. The number of communicating entities consequently becomes enormous, **21.1 billion active IoT connections by the end of 2025**.

3. **From connected objects to intelligent objects**. The Negroponte quotation adds another important point. IoT is not merely attaching an identifier to an object, such as an RFID tag that tells us where the object is. IoT is about **embedding intelligence into objects**. So a device can potentially:

   (a) Observe something about its environment (e.g., temperature, humidity, motion, light, etc.)

   (b) Process information (e.g., detect patterns, make decisions, etc.)

   (c) Communicate with other devices (e.g., send data, receive commands, etc.)

   (d) Participate in a larger system (e.g., a smart home, a smart city, etc.)

   This gives us a useful progression: from ordinary objects, to connected objects, to **networked computational components**. And once we have potentially billions of these components, distributed-system techniques become essential.

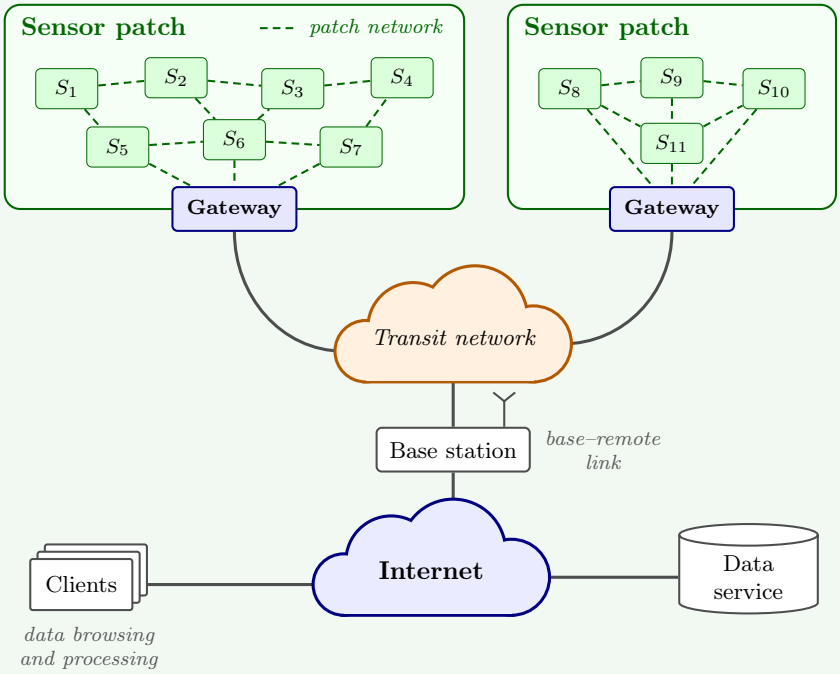
Example 3: Why a Wireless Sensor Network is a pervasive system

Imagine we want to monitor a foster, a factory, a building, or a crop field. Instead of putting one big computer in the middle, we scatter many tiny devices around the environment:

$$S_1, S_2, S_3, \ldots, S_n$$

Each one may measure something like: temperature, humidity, light, vibration, pressure, motion, etc. These devices are the **sensor nodes** and they are placed on the edge of the environment.

The crucial idea is that computing becomes part of the environment itself. We are no longer just using a computer to monitor the environment. Instead, the sensors are just "there", continuously observing the physical world. That is exactly why this is **pervasive computing**: computation is distributed throughout the environment, and it is not necessarily associated with a single device that we consciously interact with.



In the previous diagram, we represent a generic architecture for a Wireless Sensor Network (WSN).

- On the top-left, we have small devices inside the shaded region, called **sensor nodes**. Each one is a small computer with, conceptually, a CPU, memory, and a small radio transceiver. The sensor observes something physical, for example temperature, humidity, light, vibration, pressure, motion, etc.
- A group of sensor nodes may form a **Sensor Patch** (top-right), which is a small area of the environment that is monitored by a group of sensors. Inside each patch, the sensor nodes communicate through the **Patch Network**. Therefore, the sensors form a local network rather than each sensor independently connecting directly to the Internet.
- At the edge of each Sensor Patch, we have a **Gateway**. The sensor network may use some specialized local wireless technology, while the rest of the system uses a different network. The gateway bridges the two networks. So instead of each sensor node connecting directly to the Internet, the gateway collects data from the sensor nodes and sends it to the Internet.
- Multiple gateways connect to the **Transit Network**. This part simply transports information from the local sensor patches toward a more conventional infrastructure.
- The transit network reaches a **Base Station**. This is the point where data can be forwarded toward a larger network infrastruc-

ture. We call it also a **Base-Remote Link**, because it connects the local sensor network to a remote infrastructure.

- Once the information reaches the Internet, it can be sent to the **Data Service** shown on the bottom-right. This might conceptually be a database, a server, a cloud service, or an analytic system.

  For example, suppose the sensors are measuring temperature. The data service could receive: $(t_1, 22.4°C)$, $(t_2, 22.5°C)$, $(t_3, 22.6°C)$, and store or process those values.

- Finally, on the left of the Internet cloud, we have **Client Data Browsing and Processing**. That is simply the user/application consuming the collected data.

  For example, a control software could receive the temperature data and decide to turn on a fan or an air conditioner if the temperature exceeds a certain threshold.

This example shows that distributed systems are not just servers communicating over the Internet. They can also be pervasive systems, where computation is embedded in the environment itself.

Why mobility makes distributed systems harder

We can consider three different mobility scenarios that make distributed systems more challenging:

- **Normadic**. In the nomadic case, the device moves between locations, but communication is associated with whichever fixed network access point it is currently using.

  For example, a laptop may move between different Wi-Fi access points in a building. Thus, the **device changes its point of attachment** to the network, and the natural question is "***how do we continue referring to the same device when its network location changes?***". This question will become important later when we discuss *naming*.

- **Base-station mobility**. Think about a mobile phone on a cellular network. We are moving, but there is a fixed infrastructure around us. Suppose we are walking through Milan while on a call. At first our phone communicates through BS_1, then we move far enough that another base station becomes better, so we switch to BS_2.

  The important thing is that the **device moves, but the infrastructure does not**. The network has fixed access points/base stations, and the mobile node attaches to whichever one is appropriate. The distributed-system challenge is that the device's **point of attachment changes while the device itself remains the same entity**. So again, identity and current location must be kept separate.

- **Ad-hoc mobility**. Imagine several laptops or phones in the same area, and they can communicate directly with one another through their wireless

interfaces.

$$A \leftrightarrow B \leftrightarrow C$$

If A cannot directly reach C, but B can reach both, then B may conceptually act as an intermediate node. The important feature is that the devices themselves form the network. There is no fixed base station organizing the connectivity.

For example, imagine several rescue teams entering an area where there is no working network infrastructure. Their devices may form a temporary network among themselves:

$$A \leftrightarrow B \leftrightarrow C \leftrightarrow D$$

Then people move. Maybe B leaves the area, so the topology changes:

$$A \leftrightarrow C \leftrightarrow D$$

Or perhaps A can no longer reach the others at all. That is why ad-hoc networks are especially dynamic: the **network topology itself changes as the nodes move**.

The three cases can be summarized as follows:

Model	Device moves?	Fixed infrastructure?
Nomadic	Yes, between attachment points.	Yes
Base-Station	Yes	Yes, explicitly through base stations.
Ad-Hoc	Yes (potentially)	No, fixed base station.

Even without mobility, a distributed system already needs to know:

- ***Who is the other component?*** (identity)
- ***How can I reach it?*** (location)

With mobility, the second answer may continuously change.

Key Takeaways 1: Why Distributed Systems?

- Most modern applications are inherently **distributed**: their computation, data, or users span multiple communicating machines.
- Distribution is often unavoidable because of **scale**. Services such as large Web platforms and LLM infrastructures cannot realistically be provided by a single machine.
- A distributed system may involve anything from a few personal devices to millions of servers. Therefore, **distribution is not defined by geographical distance or system size**.

- Large infrastructures illustrate an important recurring idea:

  Many independent components $\longrightarrow$ one coherent service

  The user may perceive a single service even though the implementation is spread across many machines. This anticipates the concept of **transparency**.

- At sufficiently large scale, **failures are normal rather than exceptional**. A distributed system must therefore be designed assuming that individual components may fail while the rest of the system continues operating.

- Distributed systems are not limited to the Internet or the Web. Examples include: intranets, home networks, personal area networks (PANs), Internet of Things (IoT) systems, and wireless sensor networks (WSNs).

- **Pervasive/ubiquitous computing** pushes computation into ordinary objects and environments. IoT extends connectivity from people and computers toward potentially huge numbers of **networked intelligent objects**.

- A **Wireless Sensor Network** is a representative pervasive distributed system: sensor nodes form a local patch network, which connects to a gateway, which connects to a transit network, which connects to the Internet or a data service.

  The gateway bridges the local sensor network with the larger communication infrastructure.

- **Mobility makes distribution harder** because the identity of an entity and its current network location must be separated:
  - **Nomadic**: the device moves between different attachment points.
  - **Base-station**: the mobile device communicates through a fixed infrastructure and may switch from one base station to another.
  - **Ad-hoc**: there is no fixed infrastructure; the mobile nodes themselves form the network, so the network topology may change as they move.

1.1.2 Why Distribute a System?

Economics

Historically, computing was often organized around large centralized machines, such as **mainframes** (i.e., a single computer that serves multiple users), with many users or terminals depending on that central machine. The alternative is to use many smaller machines $M_1, M_2, M_3, \ldots$ and let them cooperate. The economic argument is based on the **price/performance ratio**.

Instead of buying one extremely powerful machine, it may be more convenient to buy several cheaper machines and combine their resources. The idea is not that several small computers automatically behave like one large computer. We need software that coordinates them. But **distribution allows us to exploit the aggregate resources of many machines**. So the economic motivation is that it can be cheaper to obtain computing power by combining multiple machines. This is one of the historical reasons why distributed computing became attractive.

Incremental growth and load balancing

This is one of the most important practical advantages. Suppose a centralized system starts with one machine M_1 and its workload increases. Eventually:

$$\text{load} > \text{capacity of } M_1$$

With a distributed architecture, we can add more machines:

$$M_1 \to M_1 + M_2 \to M_1 + M_2 + M_3 \to \ldots$$

THis is **incremental growth**. We do not necessarily need to replace the entire system with a much larger machine every time demand increases. Instead, capacity can grow by adding resources. This is closely related to **load balancing**.

Imagine that three machines are available:

$$M_1, M_2, M_3$$

And requests arrive:

$$R_1, R_2, R_3, R_4, R_5, R_6$$

Rather than sending everything to M_1:

$$R_1, R_2, R_3, R_4, R_5, R_6 \to M_1$$

We can distribute them:

$$R_1, R_2 \to M_1, \quad R_3, R_4 \to M_2, \quad R_5, R_6 \to M_3$$

The objective is to use the available resources more uniformly.

So there are really two connected ideas:

- **Incremental growth**: add resources gradually as the system grows.
- **Load balancing**: spread the workload among the available resources.

In summary, it makes easier to **evolve the system** and **use its resources uniformly**.

Inherent distribution

Sometimes we distribute a system because we want the benefits of distribution. But in other cases, distribution is not really a design choice, but rather the problem itself is already distributed.

For example, consider a bank. Its users, branches, ATMs, databases, and services may naturally exist in different locations: ATM Milan, ATM Rome, ATM New York, etc. All of them interact with the same overall banking system. Trying to force everything into one physical place would not make sense (i.e., it would be inefficient and impractical). This is what **inherent distribution** means: the entities involved in the problem are naturally located in different places.

Reliability

Distribution can also improve **reliability**. Consider a purely centralized architecture client-server. If that server fails, the entire service may disappear. This is a **single point of failure**. Now suppose the system is distributed over several components $S_1, S_2, S_3, \ldots$. If one of them fails, the others may still be able to provide the service. So one failure does not necessarily bring down the whole system. This is particularly important for **mission-critical applications**, where complete service failure is unacceptable.

But there is an important subtlety. Distribution does **not automatically guarantee reliability**. It merely gives us the possibility of designing a system that survives individual failures. For example, if all machines depend on one central database, then the database may still be a single point of failure. Thus, to obtain reliability, we often need techniques such as **redundancy**, **replication**, **failure detection**, and **recovery**. These are all topics that will be covered in later sections. However, here the main point is that distribution allows failures to be partial rather than necessarily total. That sounds good, and it often is, but partial failures will also turn out to be one of the hardest aspects of distributed systems.

The complexity cost of distribution

Obviously, distribution is not free. Distribution systems are considerably more difficult to get right. Everything we have discussed sounds attractive: lower cost, easy growth, load balancing, reliability, and so on. But none of this is free.

Consider two components A and B on two different machines. If A needs something from B, communication now happens through a netork:

$$A \xrightarrow{\text{network}} B$$

That introduces many questions:

- *What if the message is delayed?*

$$A \xrightarrow{\text{delayed}} B$$

- *What if the message is lost?*

$$A \xrightarrow{\text{lost}} \not B$$

- *What if B has crashed?*
- *What if B is alive but just very slow?*

From A's perspective, slow network and failed machine may be difficult to distinguish. Then there are **synchronization problems**.

Suppose two machines modify the same logical data:

$$A : x \leftarrow 5$$
$$B : x \leftarrow 7$$

Which value should the system consider correct? There is also no single perfectly shared notion of time:

$$\text{clock}_A \neq \text{clock}_B$$

And components may fail independently.

So distributing a system trades one kind of difficulty for another. A centralized system may be easier to reason about one machine at a time, while a distributed system gives us additional capabilities (many machines) but also introduces new problems: communication, concurrency, partial failure, coordination, and so on. This is the central trade-off.

Key Takeaways 2: Why Distribute a System?

There are four main motivations for distributing a system:

1. **Economics**. Several smaller machines may provide a better **price/performance ratio** than one extremely powerful centralized machine.

   many cheaper machines $\longrightarrow$ aggregate computational resources

2. **Incremental growth and load balancing**. A distributed system can grow gradually by adding resources as the workload increases:

   $$M_1 \rightarrow M_1 + M_2 \rightarrow M_1 + M_2 + M_3 \rightarrow \ldots$$

   The workload can then be distributed among those resources rather than concentrating it on a single machine.

3. **Inherent distribution**. Sometimes the problem itself is naturally distributed: users, resources, or activities already exist at different locations.

   $$\text{distributed problem} \Longrightarrow \text{naturally distributed solution}$$

   Typical examples are banking systems, reservation systems, distributed games, collaborative applications, and mobile applications.

4. **Reliability**. Distribution makes it possible for the system to continue operating even if one component fails:

   $$\text{failure of one component} \not\Rightarrow \text{failure of the entire system.}$$

   This is especially important for **mission-critical systems**. However, reliability is not automatic: the architecture must avoid introducing new single points of failure.

So in general, distribution brings advantages, **but** it also makes systems harder to build correctly. Communication, coordination, concurrency, and failures introduce complexity that does not appear in the same form in centralized systems.

1.1.3 Why Distributed Systems Are Special

⚖ Distributed vs. Centralized Systems

Start from the simplest possible centralized program: an **application running on a single machine**. Its components share the same machine, operating system, memory hierarchy, and physical environment.If one part of the program invokes another function, conceptually we have:

$$A \xrightarrow{\text{function call}} B$$

And both A and B are local (i.e., on the same machine).

Now **distribute** those components A and B with a network between them:

$$A \xrightarrow{\text{network}} B$$

That apparently small change is enormously significant. A local interaction may involve something like a function call, while a distributed interaction requires:

1. Serialization of data into a message
2. Communication over a network
3. Network transmission, which may be delayed or lost
4. Reception of the message by B
5. Deserialization of the message into data
6. Processing of the data by B

And every one of those steps introduces additional uncertainty and potential failure. This is why a distributed system is not simply a centralized program running on several machines. The network becomes part of the computation.

❓ Why building distributed systems is harder

As we saw in the previous comparison, distributed systems are more complex than centralized systems. Not only is **building them harder**, but **building them correctly is much harder**.

In general, it is often relatively easy to make a distributed application work when everything behaves normally. For example:

$$A \to B, \quad B \to A$$

Request arrives, reply arrives, everything is fine. The **difficulty** appears when reality interferes. Suppose A sends a request to B:

```
transfer(100 €)
```

B receives the request and performs the operation. Then B sends a reply back to A, but it is lost in the network. *What should A do?*

- If it sends the request again, perhaps the transfer happens **twice**.
- If it does not retry, perhaps the first request had actually been lost and the transfer happens **zero times**.

So even something that sounds trivial, like "*retry when there is an error*", may not be trivial at all. The problem is that different machines have only **partial knowledge** of what happened.

The Eight Fallacies of Distributed Computing

The **Eight Fallacies of Distributed Computing**, associated with Peter Deutsch, are assumptions that programmers are tempted to make when first designing distributed software:

1. The network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. The network is secure
5. Topology doesn't change
6. There is one administrator
7. Transport cost is zero
8. The network is homogeneous

They are called "*fallacies*" because **none of these assumptions is generally safe**. Let's analyze each of them in turn:

1. **The network is reliable**. A common mental model is:

   $$A \xrightarrow{\text{message}} B$$

   Therefore B receives the message. But in reality, this is not necessarily true. Packets can be lost, corrupted, delayed, duplicated, or delivered after a connection failure. So the actual model is closer to:

   $$A \xrightarrow[\text{maybe}]{\text{network}} B$$

   This means distributed software often needs mechanisms such as acknowledgments, retries, duplicate detection, or replication. However, the important principle is:

   $$\text{sending} \neq \text{receiving}$$

2. **Latency is zero**. A local memory access or function call is extremely fast compared with network communication. If A communicates with B:

   $$A \xrightarrow[\Delta t]{} $$

   Where Δt is the time it takes for the message to travel from A to B, which is $\Delta t > 0$, and it may vary substantially.

   A remote operation can therefore never safely be treated exactly like an ordinary local operation. For example, imagine writing:

```
for each item:
    remoteServer.get(item)
```

If every operation requires a network round-trip, latency accumulates:

$$T \approx n \cdot RTT$$

Where T is the total time, n is the number of items, and RTT is the round-trip time (i.e., the time it takes for a message to go from A to B and back). So architecture and algorithms must take communication latency seriously.

3. **Bandwidth is infinite**. A network can move only a finite amount of data per unit time:

   $$\text{bandwidth} < \infty$$

   If many machines exchange large amounts of information, the network itself may become the bottleneck.

   Imagine 1000 machines constantly sending gigabytes of state to each other. Even if every CPU is mostly idle, the system can still perform badly because the **network capacity is a resource**. This is one reason distributed algorithms often try to reduce communication.

4. **The network is secure**. Once communication crosses a network, additional attack surfaces appear. Data may potentially be intercepted, modified, or an attacker may impersonate another component.

   So communication may require authentication, encryption, integrity protection, authorization. This fallacy connects directly with the security challenge that will appear later in the notes.

5. **Topology doesn't change**. Suppose our system contains: A, B, C, and D connected in a line. It is easy to design an algorithm assuming that this arrangement will remain stable. But **distributed systems change**. A node can **fail**:

   $$A - B - \not{C} - D$$

   A new node can join:

   $$A - B - C - D - E$$

   A mobile device can move, a router can disappear, or a service can migrate. Therefore the set of nodes and connections is dynamic. This becomes especially obvious in the mobile and ad-hoc systems we saw in the previous section (page 11).

6. **There is one administrator**. On our own computer, one administrative authority can generally configure the whole system. Instead, large distributed systems may cross organizations, departments, companies, cloud providers, countries. There may be no single administrator who controls everything. Different domains may have different policies, security rules, configurations, software versions. So distributed systems often have to **operate across administrative boundaries**.

7. **Transport cost is zero**. Communication has a cost. That cost can include:

$$\text{time} + \text{bandwidth} + \text{CPU} + \text{energy} + \text{possibly money}$$

Consider two implementations.

- **Version A**. Send 1000 remote messages:

$$1000 \times \text{network operation}$$

- **Version B**. Aggregate the information and send one message:

$$1 \times \text{network operation}$$

Even if they compute logically the **same result**, their **performance** can be **dramatically different**. So communication is not free. This idea will appear repeatedly throughout distributed systems.

8. **The network is homogeneous**. A distributed system may combine machines with completely different characteristics: x86 server, ARM phone, IoT sensor, cloud VM, using different hardware, operating systems, network technologies, data representations, programming languages. Therefore distributed systems are inherently heterogeneous.

What the Eight Fallacies of Distributed Computing mean for us? We should not memorize them merely as 8 sentences. They are really **8 warnings** against pretending that:

$$\text{remote} \equiv \text{local}$$

A remote component differs fundamentally from a local one because the **network lies between us and it**. So a good distributed-system design starts from almost the opposite attitude: **assume that the network is unreliable, slow, and costly.** And then builds guarantees **on top of that unreliable and changing environment**.

Key Takeaways 3: Why Distributed Systems Are Special

A distributed system is fundamentally different from a centralized one because components communicate through a **network**:

$$\boxed{\text{local interaction} \neq \text{remote interaction}}$$

A network introduces delay, limited capacity, failures, changing topology, security concerns, and heterogeneity. Consequently, distributed systems are:

more complex to design, harder to build, and much harder to build correctly.

The Eight Fallacies of Distributed Computing summarize the assumptions that must **not** be made when designing a distributed system:

1. The network is reliable.
2. Latency is zero.
3. Bandwidth is infinite.
4. The network is secure.
5. Topology does not change.
6. There is one administrator.
7. Transport cost is zero.
8. The network is homogeneous.

They can be interpreted as four broad warnings:

- **Communication is uncertain**. Sending a message does not imply that it is received, and delays may be unpredictable.
- **Communication is not free**. Remote operations have latency, consume bandwidth, and require additional processing and resources.
- **The environment changes**. Nodes, links, locations, administrators, and network topology may change over time.
- **Distributed systems are heterogeneous**. Components may use different hardware, operating systems, networks, protocols, and software environments.

Therefore, a fundamental distributed-systems principle is:

do not design a remote interaction as if it were local

Correct distributed software must explicitly deal with communication, concurrency, uncertainty, and failures instead of assuming them away.

1.2 What is a Distributed System?

There is no single definition of a distributed system. Here we provide **three different definitions**, each of which emphasizes a different aspect of distribution.

> **Definition 1: Tanenbaum & Van Steen – The user's perspective**
>
> A **Distributed System** is a collection of independent computers that appears to its users as a single coherent system.

In the first definition, Tanenbaum & Van Steen emphasize two parts, and **both matter**:

- "***Collection of independent computers***". The system is physically composed of multiple autonomous machines:

  $$C_1, C_2, \ldots, C_n$$

  Each machine has its own processor, memory, operating system, and can potentially fail independently.

- "***Appears to its users as a single coherent system***". Externally we want:

  $$C_1 + C_2 + \ldots + C_n \approx \text{one coherent system}$$

  The user should not necessarily have to think: "*my request is currently being handled by machine 37 in data center B*". Instead, the user sees something like *Google Search* or *online banking service* even though internally many machines cooperate.

So this definition combines two seemingly contradictory ideas:

physical multiplicity and **logical unity**

In other words, many machines underneath, one system from the outside. It anticipates the concept of **transparency**, which we will see later.

> **Definition 2: Lamport – The failure perspective**
>
> A **Distributed System** is a system on which I cannot get any work done because some machine I have never heard of has crashed.

It sounds like a joke, but it highlights one of the **deepest properties** of distributed systems. Imagine that we are using service A. From our point of view, perhaps A is working perfectly. But internally:

$$A \to B \to C \to D$$

And A depends on B, which depends on C, and so on. Now suppose that C crashes. Then B cannot complete its work, and A cannot complete its work either:

$$C \text{ fails} \implies B \text{ cannot proceed} \implies A \text{ cannot serve users}$$

So our application fails because of a remote component completely invisible to us. This reveals an essential property: components fail independently. A centralized system often has a simpler failure model: if the central server fails, the whole system fails. In a distributed system, failures can be more subtle and complex:

$$C_1 : \checkmark \qquad C_2 : \checkmark \qquad C_3 : \times \qquad C_4 : \checkmark$$

The system is neither completely alive nor completely dead. This is what we later call a**partial failure**. Therefore, Lamport's definition emphasizes that distribution creates dependencies on independently failing components.

Definition 3: Coulouris – The communication perspective

A **Distributed System** is a system in which hardware or software components located at networked computers communicate and coordinate their actions only by passing messages.

Suppose we have two processes P_1 and P_2 running on different machines. They do **not** communicate by directly reading and writing the same physical memory. Instead, they communicate by sending messages over the network:

$$P_1 \xleftrightarrow{\text{network}} P_2$$

Communication and coordination occur through the network. So if P_1 wants information held by P_2, it cannot simply do:

$$x = \text{memory}\,[\texttt{address}]$$

It needs some form of communication:

$$\text{request} \to \text{network} \to P_2$$

Followed potentially by:

$$P_2 \to \text{network} \to \text{reply}$$

This immediately explains why the network matters so much. However, the network introduces a new source of complexity: delay, message loss, finite bandwidth, failures, uncertainty, and so on. Therefore, message passing is not merely an implementation detail, it fundamentally changes how programs must be designed.

Therefore, the 3 definitions are not competing definitions, they describe the same object from different perspectives:

- **Structural viewpoint**. Multiple independent computers that should appear as one coherent system.
- **Failure viewpoint**. Independent components can fail and affect one another.
- **Communication viewpoint**. Components coordinate through message passing.

Together, we obtain a very useful working definition.

Definition 4: Distributed Systems

A **Distributed System** consists of independent networked components that coordinate through message passing to provide some coherent overall service.

Independent does not mean isolated. When we say "*independent networked components*", we do **not** mean that C_1 and C_2 are completely isolated with no interaction.

If they were completely isolated, we would simply have two unrelated computers. Instead, we mean that they are **autonomous**, in the sense that each component has its own processor, memory, and operating system, but they can **cooperate** to provide a service.

Parallel Computing vs. Distributed Computing

A natural question is: ***what is the difference between parallel computing and distributed computing?*** At first sight, the two fields look almost identical.

- **Parallel computing** P_1, P_2, …, P_n perform computation together.
- **Distributed computing** C_1, C_2, …, C_n also perform computation together.

So why distinguish them? Because the **assumptions about the architecture are different**.

How tightly coupled are the processors? A useful way of understanding the distinction is through **memory and communication**. Consider two processors CPU_1 and CPU_2:

- **Shared-Memory Parallel System**. The two processors may **share the same shared memory**. Then both processors can access variables stored in that memory.

  If $x = 5$ is stored in shared memory, both processors can potentially read x. Communication can therefore happen implicitly through memory:

  $$\text{CPU}_1 : \; x \leftarrow 5 \quad \text{and} \quad \text{CPU}_2 : \; \text{read } x$$

  This is characteristic of a **tightly coupled** parallel machine.

- **Distributed Systems use Private Memories**. Now consider two separate computers:

$$\text{CPU}_1 + \text{Memory}_1 \quad \text{and} \quad \text{CPU}_2 + \text{Memory}_2$$

  There is no physical shared memory between them. Communication can only happen through message passing over the network:

$$\text{CPU}_1 \overset{\text{network}}{\longleftrightarrow} \text{CPU}_2$$

So the conceptual difference is that parallel computing assumes shared memory, while distributed computing assumes private memories but networked communication. This is exactly why the Coulouris definition stresses the sentence: "***only by passing messages***".

Furthermore, there is another architectural dimension: **how processors are interconnected**. A simple multiprocessor may use a common bus to connect all processors with shared memory. This makes communication very direct, but a common bus does not scale indefinitely because all processors compete for the same interconnect.

⚠ More sophisticated parallel machines may therefore use switching networks connecting processors and memories. Thus, *parallel computing is similar to distributed computing, right?* Not quite. The essential point for us is not to memorize every hardware topology, but to understand that parallel computing often assumes a much tighter integration between processors than a typical distributed system.

⚖ **Different primary objectives.** There is also a useful conceptual distinction in **why** we use multiple processors. In **parallel computing**, the central goal is often to **speed up one computation**. Suppose a computation takes time T on one processor, ideally with p processors we would like:

$$T_p \approx \frac{T}{p}$$

Although real systems obviously have overhead. Instead, **distributed computing** has broader goals: scalability, geographical distribution, resource sharing, reliability, inherent distribution of users/data. Performance can certainly be one objective, but it is **not the defining one**.

- Parallel computing: mainly cooperation for computational performance.
- Distributed computing: cooperation among autonomous networked components.

Key Takeaways 4

- **A distributed system is made of independent computers/components that cooperate to provide one coherent**

system.

$$\boxed{\text{many independent machines} \rightarrow \text{one logical service}}$$

- **Communication and coordination happen through message passing.** Components on different machines cannot simply access each other's memory:

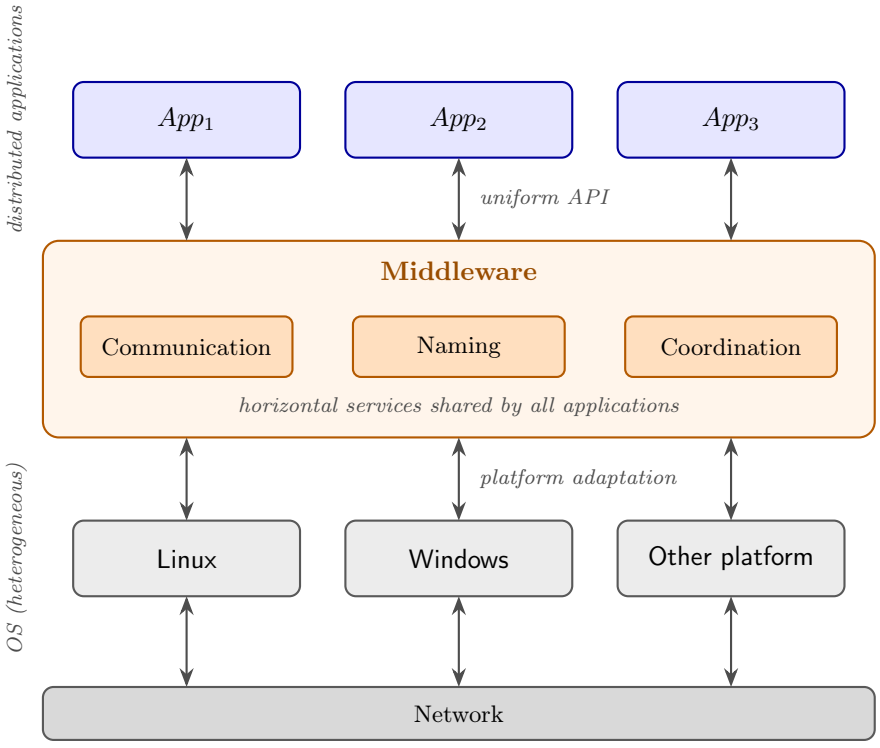
$$P_1 \xrightarrow{\text{message}} P_2$$

- **Independence implies independent failures.** One machine may fail while the others continue running. This possibility of **partial failure** is one of the defining difficulties of distributed systems.
- **Distributed computing is not the same as parallel computing.** Both use multiple processors, but parallel systems are generally more tightly coupled and may use shared memory, whereas distributed systems consist of autonomous machines with local/private state connected through a network.
- **The main objective differs.** Parallel computing is primarily concerned with accelerating computation; distributed computing is broader and includes cooperation, scalability, geographical distribution, resource sharing, and reliability.

A distributed system consists of autonomous networked components that coordinate by message passing while appearing as one coherent system.

1.3 Middleware

Middleware is the **software layer between the operating system/ network and the distributed application**.



Middleware hides heterogeneity;
it does not eliminate the network.

Figure 1: Middleware as a common layer between distributed applications and heterogeneous platforms.

The key idea is that applications should not have to deal directly with every low-level difference between machines, operating systems, and network mechanisms.

Why middleware exists

Suppose we have two applications running on different machines: A and B. Machine A could use one operating system, while B uses another. They may expose different APIs, use different data representations, or provide different networking facilities.

- **Without middleware**, the application itself has to handle many of these differences:

  $$\text{Application} \rightarrow \text{OS/network-specific details}$$

  This makes distributed applications harder to build and less portable.

- **Middleware** introduces an intermediate abstraction:

$$\text{Application} \rightarrow \boxed{\text{Middleware API}} \rightarrow \text{OS/network}$$

  So that the application can interact with a **more uniform interface**.

The central purpose is to hide some platform heterogeneity from distributed applications.

Network/OS-based distributed system

One possibility is to build the distributed application directly on top of the networking and operating-system facilities. In this case, the application may need to deal directly with things such as: sockets, network addresses, data encoding, communication details, and differences between platforms. This gives the programmer a lot of control, but also exposes a lot of **complexity**.

Middleware-based distributed system

With middleware, we introduce another layer between the application and the operating system/network. The application uses the services provided by middleware rather than directly implementing everything using low-level network primitives. This has **two main purposes**:

1. **Providing horizontal services**. Middleware provides **horizontal services** useful to many different distributed applications. "*Horizontal*" means that the service is not specific to one particular business application. For example, conceptually, different applications may all need common mechanisms such as communication, naming, and coordination, rather than implementing them independently every time.

   So instead of:

$$App_1 \rightarrow \text{implement everything}$$

$$App_2 \rightarrow \text{implement everything again}$$

   we obtain:

$$\begin{array}{ccc} App_1 & App_2 & App_3 \\ \downarrow & \downarrow & \downarrow \\ & \boxed{\text{Middleware}} & \end{array}$$

   The common distributed functionality is **reusable**.

2. **Masking platform differences**. This is the second explicit role of middleware. Imagine a linux machine, a windows machine, and another platform (e.g., MacOS, Android, etc.). At the lower level, these systems may be quite **different**. Middleware tries to offer the applications a common abstraction, while internally adapting to each platform. So middleware acts somewhat like an **adapter layer**.

Notice that it does not magically make the underlying machines identical. It simply **masks some of the differences** so that application developers do not have to handle all of them directly.

Example 4: Middleware hides complexity

Suppose application A wants to invoke functionality provided by application B.

✗ Without a middleware abstraction, A might have to think in terms of:

$$\text{IP address} + \text{socket} + \text{message format} + \text{network communication}$$

✓ With middleware, the application can instead work with a higher-level operation:

$$A \rightarrow \boxed{\text{middleware service}} \rightarrow B$$

The low-level communication still happens, of course. Middleware does **not remove the network**, it just hides part of its complexity behind a higher-level abstraction.

Key Takeaways 5

Just three key points to remember about middleware:

1. Middleware sits between applications and the OS/network.
2. It provides reusable horizontal services for distributed applications.
3. It masks platform differences, making applications easier to build.

So in one sentence: Middleware is a common software layer that simplifies distributed programming by providing shared services and hiding platform heterogeneity.

1.4 Fundamental Properties

In this section we discuss three fundamental properties of distributed systems. More precisely, these are three characteristics that arise from the fact that a distributed system consists of **independent computers communicating over a network**: concurrency, no global clock, independent/partial failures. These properties are consequences of distribution itself.

Concurrency

In a centralized system, concurrency is often a **design choice**. We may decide to write a single-threaded program, or a multi-threaded program. In a distributed system, instead, concurrency is essentially **unavoidable**. Suppose we have tre independent computers: C_1, C_2, and C_3. Each has its own processor and executes independently:

$$C_1 : a_1 \to a_2 \to a_3$$
$$C_2 : b_1 \to b_2 \to b_3$$
$$C_3 : c_1 \to c_2 \to c_3$$

Their computations are happening **at the same time**. Therefore, concurrency is inherent in distributed systems. This creates **coordination problems** whenever several components access or modify logically shared resources, like **synchronization**, **mutual exclusion**, and **consistency**.

Absence of a Global Clock

On one computer, we have a physical clock that the operating system can use as a common time reference. Different processes can therefore consult approximately the same underlying clock.

In a distributed system, however, each computer has its own physical clock:

$$C_1 \to \text{clock}_1$$
$$C_2 \to \text{clock}_2$$
$$C_3 \to \text{clock}_3$$

And these clocks are **not necessarily synchronized**:

$$\text{clock}_1 \neq \text{clock}_2 \neq \text{clock}_3$$

Even if we initially synchronize them, physical clocks drift. So there is no perfectly shared **global clock** available to every component.

This becomes especially problematic when we want to determine the **ordering of events**. Looking only at the local timestamps may not always be sufficient, because the clocks can disagree. So distributed systems need other ways to reason about ordering and synchronization. Later in this notes we will discuss **logical clocks** rather than physical clocks. However, in general, there is no perfectly shared notion of time across all machines.

Independent/Partial Failures

In a centralized system, if the machine crashes, typically the whole application fails. The failure is relatively easy to conceptualize.

In a distributed system, instead, each machine can fail independently:

$$C_1 : \checkmark \qquad C_2 : \checkmark \qquad C_3 : \times \qquad C_4 : \checkmark$$

One component fails while the others continue running. As anticipated in the previous section, this is called a **partial failure**.

Also, communication makes failure detection more difficult. Suppose:

$$A \xrightarrow{\text{request}} B$$

And A receives no response. What happened?

1. B crashed
2. The request was lost
3. The reply was lost
4. B is simply slow
5. The network is extremely slow or unavailable

For A, all of these can initially appear as no response from B. Therefore determining whether another component has actually failed can be **hard or even impossible with certainty** under some assumptions. This is a much deeper problem than in an ordinary centralized application.

Finally, recovery is also harder. Suppose machine C_2 crashes and later restarts. *Where is the application state?* It may be distributed across C_1, C_2, and C_3. Perhaps C_1 believes one thing while C_3 has slightly different information. So recovery is not simply a matter of restarting the crashed machine. We may need to reconstruct a **consistent distributed state**. This is why fault tolerance becomes such a large and important topic in distributed systems.

> **Key Takeaways 6**
>
> Distributed systems inherently execute **concurrently**, have **no perfectly shared global clock**, and can suffer independent partial failures. These three properties are fundamental because most of the difficult problems in distributed systems arise from them.

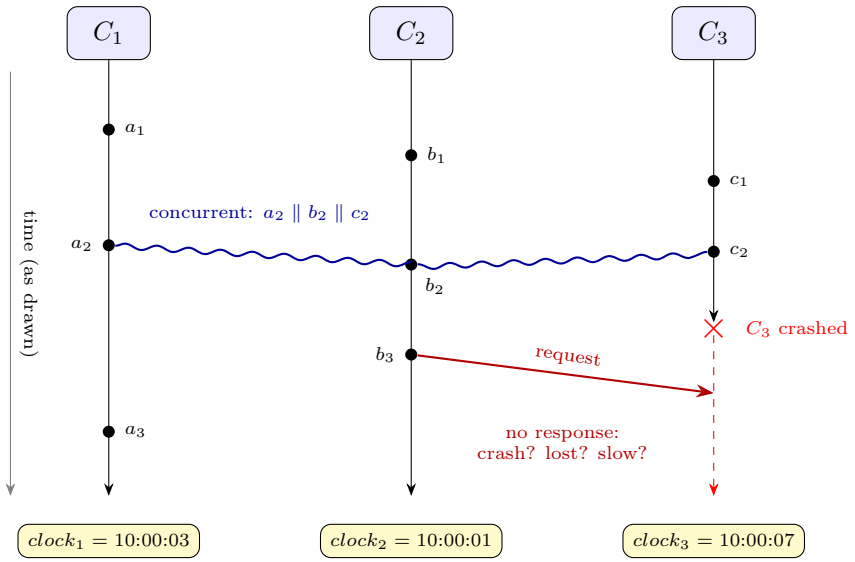


Figure 2: **The three fundamental properties of distributed systems.** Independent computers C_1, C_2, C_3 execute concurrently (blue wavy line: no causal path between the events), each has its own physical clock with no global time (yellow boxes), and a component such as C_3 can fail while the others keep running, leaving the sender unable to tell a crash from a lost message or a slow node (red).

1.5 Main Challenges

The three fundamental properties we just discussed (concurrency, no global clock, partial failures) tell us **why distributed systems are intrinsically difficult**. Now we move to the practical engineering question: *what problems must we solve when actually building one?* We can identify seven main challenges: heterogeneity, openness, security, scalability, failure handling, concurrency, and transparency.

In this section, we will discuss the first six challenges, because they try to **cope the difficulties created by distribution**. Instead, transparency, even though it is a challenge, is more of a **design objective** because it decides how much of the underlying distribution is exposed to the user or programmer. We will discuss transparency in next section.

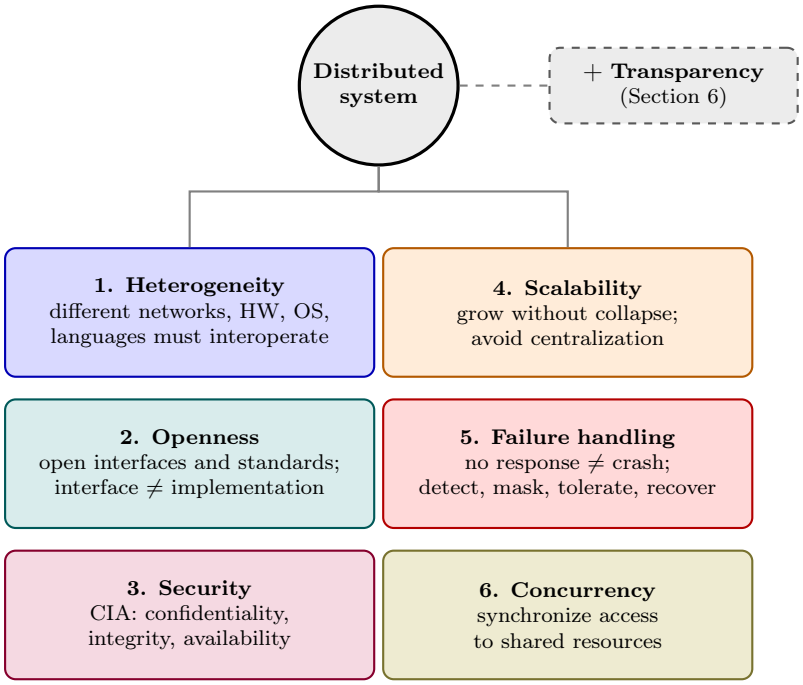


Figure 3: Main challenges of distributed systems.

Heterogeneity

Heterogeneity means that the components of a distributed system may be fundamentally different from one another. A distributed system may connect different networks, hardware, operating systems, programming languages, and so on. Yet all of them are expected to communicate correctly. That is the **challenge**.

- **Network heterogeneity**. Even if everything uses TCP/IP, the underlying networks can be very different. For example, wired Ethernet, wireless Wi-Fi, wireless ad-hoc networks. TCP/IP can hide many differences at higher levels, but it cannot make the underlying physical environments identical.
- **Hardware heterogeneity**. Now suppose two machines have different architectures. For example, one is x86 and the other is ARM. Imagine one machine sends a binary representation of a value to another. If both machines interpret that sequence of bytes differently, communication breaks. Therefore, distributed communication needs **agreed representations**.
- **Operating-system heterogeneity**. Different machines may run different operating systems (e.g., Linux, Windows, macOS). The networking functionality may ultimately be similar, but the APIs and semantics exposed to applications can differ. So if a distributed application relies too heavily on one platform's behavior, **portability becomes difficult**.
- **Programming-language heterogeneity**. Suppose a three component distributed system has one component written in C, another in Java, and the third in Python. Each language has its own data types, memory management, and runtime behavior. If these components need to exchange complex data structures, we must define a common serialization format and ensure that each language can correctly encode and decode it.

So the core challenge is: ***how do heterogeneous components interoperate?*** This is precisely one of the problems middleware can help with.

Openness

Openness is about whether the distributed system can be extended, reimplemented, and integrated with other systems. The central idea is that other components must know how to interact with the system. That **requires clearly defined rules**.

- **Publish protocols and interfaces**. Suppose we build a service S. If nobody knows which messages it accepts, what those messages mean, what responses it return, then nobody else can implement a compatible client.

  So an open system needs published **protocols** and **service interfaces**. Note that same interface does **not** imply same implementation. For example, two different implementations of a web server can both support the same HTTP protocol and serve the same content, but they may have different internal architectures and optimizations.

- **Interoperability**. Suppose company A implements a client and company B implements a server. If both follow the same standard, they should be able to work together. That property is called **interoperability**. Different implementations of the same protocol should be able to communicate correctly, even if they are built by different teams, in different languages, and on different platforms.

- **Portability**. Openness should also help software move between environments. For example, if an application depends on a standardized abstraction rather than proprietary low-level details, it becomes easier to move it from one platform to another. So:

  - **Interoperability** concerns different systems working together;
  - **Portability** concerns moving software between environments.

- **Standards**. To achieve openness, standards are extremely useful. Instead of every organization inventing its own protocols, we agree on **common standard**. Then independent implementations can cooperate.

  The internet is the classic example. Internet protocols are specified through **RFCs**, and the **IETF** provides an open standardization process. This is one major reason completely different devices can communicate over the internet.

We can summarize the openness idea as: specify what components must do, not how they must be implemented.

Security

Distribution greatly increases the **security problem** because resources are accessed through a network. In a centralized machine, many resources may be protected by the physical and operating-system boundary. In a distributed system:

$$\text{Client} \xrightarrow{\text{network}} \text{Server}$$

And potentially anyone connected to that network may attempt to interact with the service.

The three main security properties that we want to achieve are called the **CIA triad**:

$$C = \text{Confidentiality}, \qquad I = \text{Integrity}, \qquad A = \text{Availability}$$

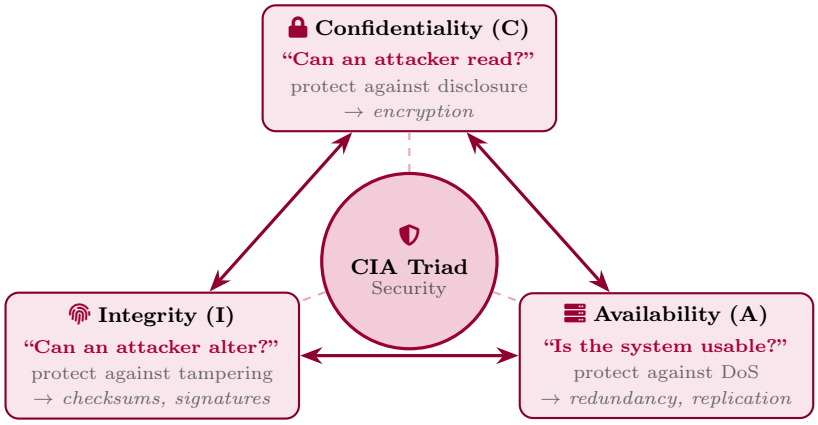


Figure 4: The CIA triad: core security objectives in distributed systems.

Where:

- **Confidentiality** means that information is accessible only to authorize entities.

  > **Example 5: Confidentiality**
  >
  > Suppose Alice sends a *password* to a remote server. Without protection, an attacker could intercept the message and read the password. We therefore want that an unauthorized observer cannot learn the content of the message.
  > So confidentiality protects against **unauthorized disclosure**. Encryption is one major mechanism used to achieve this.

- **Integrity** means protecting information against **unauthorized alteration or corruption**.

> **Example 6: Integrity**
>
> Suppose Alice sends a message "`transfer 10€`" to Bob. An attacker should not be able to transform it into "`transfer 1000€`" without being detected.
> So confidentiality ask "*can an attacker read the information?*", but integrity asks "*can an attacker change the information?*".

- **Availability** means that legitimate users must be able to access the resource.

> **Example 7: Availability**
>
> An attacker may not steal or modify anything and still successfully attack the system by preventing others from using it. For example, this is the idea behind **Denial of Service (DoS)** attacks, where the attacker floods a server with requests so that legitimate users cannot get through.
> Suppose the server can process 1000 requests per second and an attacker generates 1 million requests per second. Legitimate requests may become impossible to serve.
> So **security is not only about secrecy**, but also about **ensuring that the system remains usable**.

Scalability

A distributed system is **scalable** if we can increase its size without causing an unacceptable degradation in performance. But "size" has more than one meaning. There are **three dimensions of scalability**:

- **Scaling users and resources**. The simplest case is increasing the number of users, machines, services, or data. A scalable architecture should not collapse simply because the number of participants grows. The fundamental challenge is therefore reconcile increasing scale with acceptable performance. It is easy to build something that works for 10 machines. The difficulty is designing it so that adding another 10,000 machines does not fundamentally break the architecture.
- **Geographic scalability**. A system may also grow geographically (e.g., from a single data center to multiple data centers across the world) and the physical distance becomes relevant. So geographic growth introduces new challenges, such as **latency** and **fault tolerance**. For example, if a system is designed to operate within a single data center, it may not handle the increased latency and potential failures that come with a global distribution.
- **Administrative scalability**. A small distributed system may be completely controlled by **one organization**. But as the system grows, it may involve multiple organizations, each with its own policies, procedures, and administrative domains. So scalability is not merely "*can we add more servers?*", but also "*can the system grow in users, geography, and administrative domains?*"

The enemy of scalability: Centralization. In general, centralized components tend to become scalability bottlenecks. There are three typical forms of centralization:

1. **Centralized services**. Suppose every user must contact one server:

   $$\begin{array}{ccccccc} C_1 & C_2 & C_3 & \cdots & C_n \\ & \searrow & \downarrow & \swarrow & \\ & & \boxed{S} & & \end{array}$$

   As n grows, server S eventually becomes overloaded, a performance bottleneck, potentially a single point of failure. So one server for everybody does not scale indefinitely.

2. **Centralized data**. Suppose the entire distributed system depends on one global directory: one online telephone book, one DNS server, one global database. Then, every operation requires querying it. Again:

   $$\text{more machines} \Rightarrow \text{more requests to the same data source}$$

   Eventually the centralized data structure becomes the bottleneck.

3. **Centralized algorithms**. Imagine an algorithm that makes decisions using the **complete state of the entire system**. For example, a routing algorithm that:

   $$\text{collect information about every node} \rightarrow \text{compute routing decision}$$

With 5 machines, obtaining complete information may be easy, but with 10^6 machines, maintaining an always-up-to-date global view can become extremely expensive or unrealistic.

So scalability requires us to be suspicious not only of centralized servers, but also of algorithms that conceptually require complete global knowledge.

Decentralized algorithms. Due to the scalability problems of centralized algorithms, we often prefer **decentralized algorithms**. There are four properties worth remembering:

1. No machine has complete information. Instead of one machine knowing everything, each machine only knows a part of the system's state.
2. Decisions use local information. A node makes decisions based on the information available to it, rather than first collecting the complete global state. This reduces coordination overhead.
3. One machine failing should not destroy the algorithm. The algorithm should be robust to failures, so that the failure of a single node does not prevent the system from functioning.
4. No implicit global-clock assumption. The algorithm should not rely on the existence of a global clock or synchronized time across machines, as this is often unrealistic in distributed systems (as discussed in previous sections).

Techniques for scalability. Several techniques can help achieve scalability:

- **Asynchronous communication**. Instead of waiting for a response, a node can continue processing other tasks. This allows the system to handle more requests concurrently.
- **Caching and replication**. Instead of forcing everybody to retrieve data from one place, we can replicate data across multiple nodes and cache frequently accessed data. Of course, replication introduces a new problem: ***how do we keep the copies consistent?*** That will become an entire topic of its own in later sections.
- **Epidemic dissemination**. The name is intentionally evocative: information spreads like a virus. Instead of one node sending information to all others, it sends it to a few neighbors, which then send it to their neighbors, and so on. This can be more efficient than flooding the network with messages.
- **Hierarchical structuring**. Instead of organizing millions of components in one flat structure, we divide them hierarchically. Then many operations can be performed locally within a sub-hierarchy, reducing the need for global coordination.

However, in general the **principle** is: global knowledge and global coordination become increasingly expensive as a system grows. Therefore scalable distributed systems tend toward **decentralization, local decisions, and distribution of data/workload.**

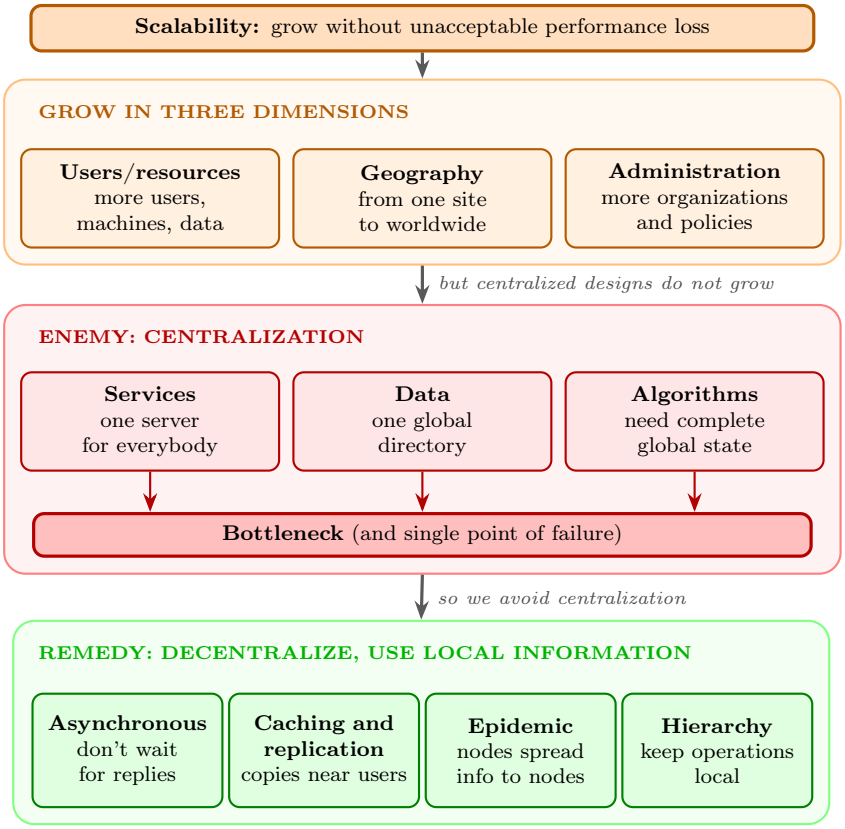


Figure 5: Scalability: dimensions, enemy and remedies.

Failure Handling

We already encountered **partial failures** as a fundamental property. Now the question becomes: "*what can the system actually do about failures?*". And partial failures are problematic precisely because some components may continue operating while others fail:

$$A : \checkmark \qquad B : \times \qquad C : \checkmark$$

The system has to decide how to behave in this intermediate state. Usually, the system applies the following four strategies:

$$\text{detection} \rightarrow \text{masking} \rightarrow \text{tolerance} \rightarrow \text{recovery}$$

Plus **redundancy**. These are related, but they are **not synonyms**.

1. **Detecting failures**. Some failures are **easy to detect**.

   > **Example 8: Easy failure detection**
   >
   > Suppose a received message is corrupted. A checksum can reveal:
   >
   > $$\text{received data} \neq \text{expected valid data}$$
   >
   > So we know a failure occurred.

   But other failures are fundamentally **harder**.

   > **Example 9: Hard failure detection**
   >
   > Suppose a client sends a request to a server, but the server does not respond. Is the server down? Or is the network just slow? Or did the response get lost? We cannot distinguish between these cases with certainty. This is a central distributed-systems issue.

   For some remote failures, we cannot detect them **with certainty**, we can only **suspect** that they occurred. So the challenge becomes manage failures that cannot always be definitively detected.

2. **Masking failures**. Sometimes, once a failure is detected, we can prevent the application from noticing it. This is called **failure masking**.

   > **Example 10: Failure masking**
   >
   > Suppose a message fails to arrive. We can retransmit and if the second transmission succeeds, the user may never notice that the first message was lost.

   So masking means hiding the effect of a failure. The failure happened, but we simply prevented it from becoming visible at the higher level.

3. **Tolerating failures**. Not every failure can be completely hidden. Sometimes the system instead continues operating despite the failure. This is called **fault tolerance**.

> **Example 11: Fault tolerance**
>
> Suppose a distributed service normally provides:
>
> $$\{A, B, C, D\}$$
>
> But one component fails. Perhaps the system can continue providing:
>
> $$\{A, B, C\}$$
>
> While temporarily losing functionality D.

Fault tolerance is based on **degraded service**, because reduced functionality is preferable to complete failure. The system may not be able to provide everything, but it can still provide something.

4. **Recovering from failures**. Eventually we may want the failed component to return to a valid state. One basic technique is periodically storing its state on permanent storage. Suppose the component evolves:

   $$S_0 \to S_1 \to S_2 \to S_3$$

   And periodically persists:

   $$S_2 \to \boxed{\text{stable storage}}$$

   Then the component crashes while in S_3:

   $$S_3 \times$$

   After restarting, we may restore:

   $$\boxed{S_2}$$

   Rather than starting from nothing. So recovery restores a component to a usable state after failure. The fact that the overall application state is distributed makes recovery considerably more **complicated** than simply restarting one process, but that deeper problem comes later.

Redundancy. One of the most common ways to mask or tolerate failures is redundancy. Instead of having:

$$\boxed{\text{Server}_A}$$

We may have:

$$\boxed{\text{Server}_A} \quad \boxed{\text{Server}_B} \quad \boxed{\text{Server}_C}$$

If:

$$\text{Server}_A \times$$

Another server may continue providing the service. This is **replicated servers**. And the same concept applies to communication paths providing **redundant routes**, or to data storage providing **replicated data**. So redundancy means: do not make the system depend on a single instance of a critical resource. But there is an important warning, because **keeping replicas in a distributed system is not easy**. If we have multiple copies of a resource, we must ensure that they remain **consistent** with one another. That bring us again toward the later topic of **consistency and replication**.

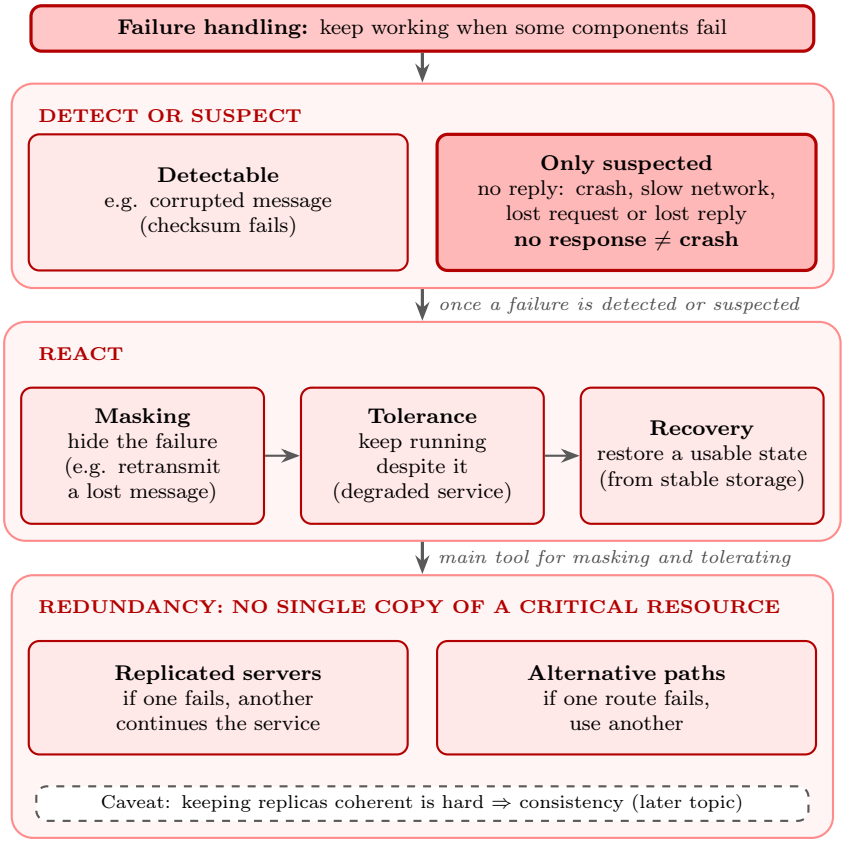


Figure 6: Failure handling: detect or suspect, react, and redundancy.

Concurrency

We already discussed **concurrency** as a **fundamental property** of distributed systems. Now it appears again as a **practical challenge**.

- Property: concurrency inevitably exists. Suppose we have multiple nodes in a distributed system. Each node may be executing its own tasks independently. Even if we try to serialize operations, the fact that nodes are physically separate means that they can still operate concurrently.
- Challenge: we must manage it correctly. Concurrency becomes **dangerous** when multiple components interact with a **shared resource**.

  Suppose two components concurrently operate on the same information:

  $$\text{Component}_A : \text{read}(x) \rightarrow \text{write}(x+1)$$

  $$\text{Component}_B : \text{read}(x) \rightarrow \text{write}(x+1)$$

  Without proper synchronization, the final result might depend on the timing of the operations. So access to shared resources must be carefully synchronized.

Key Takeaways 7

In general, different components must work together, openly, and securely, at scale, despite failures and concurrency.

- **Heterogeneity**: different networks, hardware, OSs and languages must interoperate.
- **Openness**: published protocols/interfaces and standards enable interoperability and portability.
- **Security**: guarantee **confidentiality, integrity and availability**.
- **Scalability**: support growth in **users/resources, geography and administration**; avoid excessive centralization and prefer decentralized/local decisions.
- **Failure handling**: failures may be uncertain and partial;

  $$\text{detect/suspect} \rightarrow \text{mask} \rightarrow \text{tolerate} \rightarrow \text{recover}$$

  Often using **redundancy** to avoid single points of failure.
- **Concurrency**: components execute simultaneously, so access to shared information/services must be **synchronized**.

1.6 Transparency

A distributed system should hide unnecessary aspects of distribution so that users and programmers can interact with it more simply. Transparency is largely about achieving the **illusion** that the system is a single, coherent entity. The user should not have to reason about every server, replica, physical location, migration, or failure unless that information is actually important.

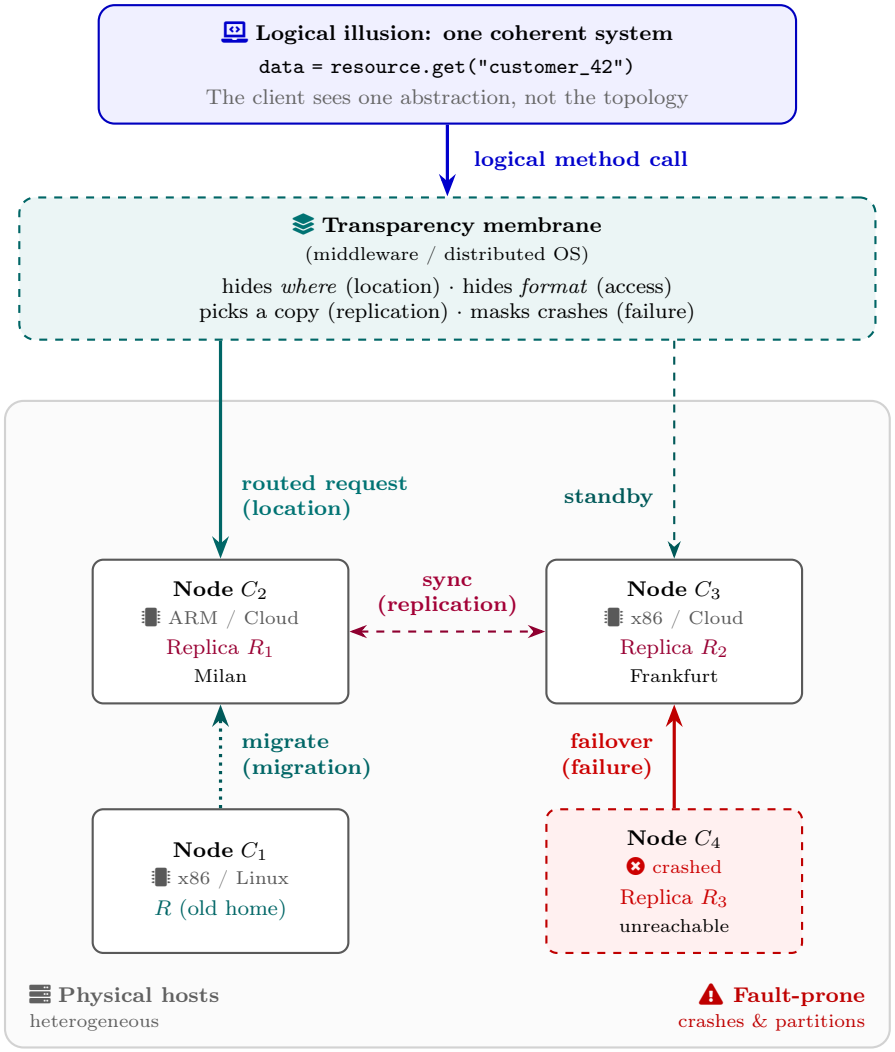


Figure 7: **Distributed-system transparency**: a logical resource is presented as a single coherent abstraction while middleware hides selected details of its physical distribution, such as location, representation, migration, replication, and failures.

In the following, we will discuss the different types of transparency that can be achieved in a distributed system when we try to answer the question: "*which part of distribution are we hiding?*".

Access transparency means hiding differences in data representation and the way a resource is accessed. Suppose one resource is local (e.g., a file on the local disk) and another is remote (e.g., a file on a remote server). Ideally, the application should not need completely different programming models just because one resource happens to be on another machine. Similarly, different machines may represent data differently, but those representation details can be hidden from the application. So access transparency **hides how a resource is accessed and represented**. Middleware is closely related to this idea.

Location transparency. Suppose we want to access resource R. Without location transparency, we might need to know that R is on machine M. Instead, with **location transparency**, we can simply refer to R without knowing where it physically resides. So location transparency **hides the physical location (*where*) of resources**.

Migration transparency. Now suppose resource R starts on machine A and later migrates to machine B. With **migration transparency**, users should not need to know that this movement occurred. They continue to refer to R in the same way. So migration transparency **hides the movement of resources**.

Relocation transparency. Migration and relocation sound almost identical, but there is a useful distinction.

- If a resource changes location and this fact is hidden from the user, we have **migration transparency**.
- If a resource changes location **while it is being used**, and this fact is hidden from the user, we have **relocation transparency**. Ideally, the client continues operating without needing to restart everything.

So relocation transparency is a stronger form of migration transparency, and it **hides the movement of resources while they are being used**.

Replication transparency. Suppose logically we have one resource R, but physically we have multiple copies of it (e.g., for fault tolerance or load balancing). The user should ideally not have to choose which copy to use, and the system should automatically select a copy for the user. This is **replication transparency**, which **hides the fact that many physical copies appear as one logical resource**.

However, replication transparency creates another problem that we have cited before: **consistency and replication**. If a user updates one copy of a resource, the system must ensure that all other copies are updated as well. Otherwise, the user may see different values for the same resource depending on which copy they access.

Concurrency transparency. Suppose two users access the same resource R at the same time. They may operate concurrently. With **concurrency transparency**, each user should ideally experience sensible

behavior without having to manually coordinate with the other user. So concurrency transparency **hides the fact that resources are simultaneously shared**.

Failure transparency. Suppose a server fails but there is another server that can take over its responsibilities. The system might automatically redirect requests to the new server without the user noticing. This is **failure transparency**, which **hides the failure and recovery of resources**.

Persistence transparency. Suppose an object exists in memory (e.g., RAM) and later it is stored on disk. The application ideally interacts with the logical resource without needing to know exactly where its current representation resides. So persistence transparency **hides whether a software resource is in memory or on persistent storage**.

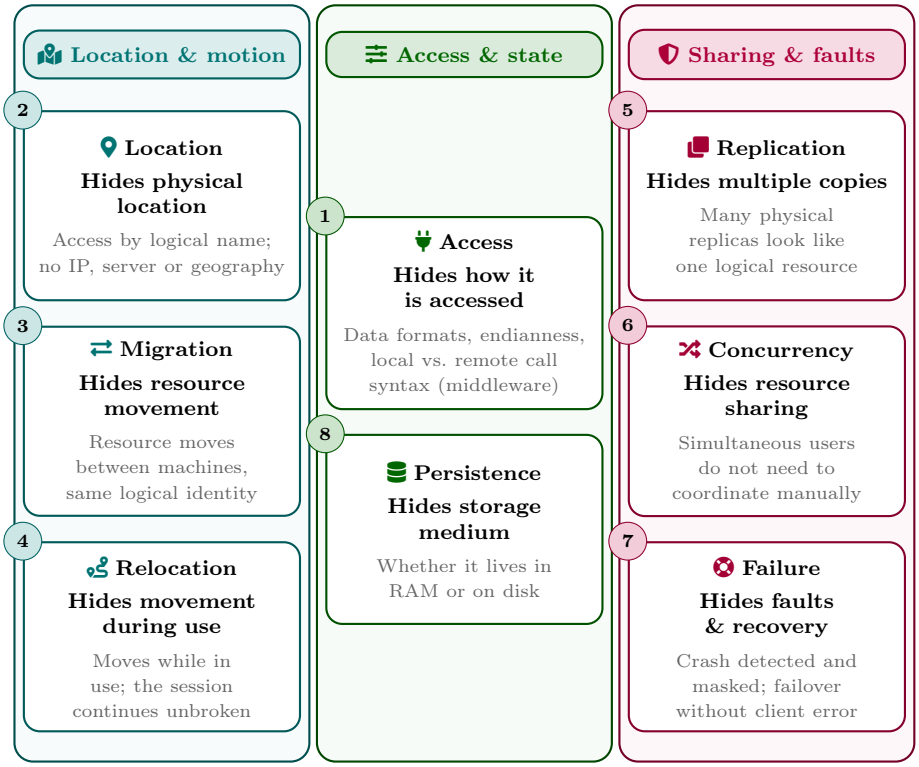


Figure 8: **Taxonomy of transparency**: the eight forms grouped by the aspect of distribution they hide.

⚠ Transparency is not always good

At first sight we might conclude that more transparency is always better. However, this is **not always** the case. There are two limits:

1. **Some things cannot really be hidden**. If a server is physically on another continent, the network delay is real. Middleware can provide a beautiful abstraction such as:

   ```
   remoteObject.method(args)
   ```

   That looks almost like:

   ```
   localObject.method(args)
   ```

   But physically, the remote call will take much longer than the local call. So we cannot abstract away physical realities. In general, abstraction can hide complexity, but it cannot remove latency.

2. **Too much transparency can hurt performance**. This is an important design lesson that is often overlooked.

   Imagine middleware makes a remote object look exactly like a local object:

   ```
   object.method(args)
   ```

   We might naturally write:

   ```
   for i in range(1000000):
       object.method(args)
   ```

   This is a very natural way to write code, but it is **terrible for performance**. The reason is that each call to `object.method` will incur a **network round-trip**, which is very slow. The programmer did not optimize the access because the system successfully hid the fact that the object was remote!

   Paradoxically, hiding distribution too well can lead to bad distributed software.

Ok, then how much should we hide? The goal is **not** to hide everything, but to hide only unnecessary complexity while exposing differences that matter for: (1) performance, (2) correctness, and (3) application behavior. In other words, good abstraction requires judgment.

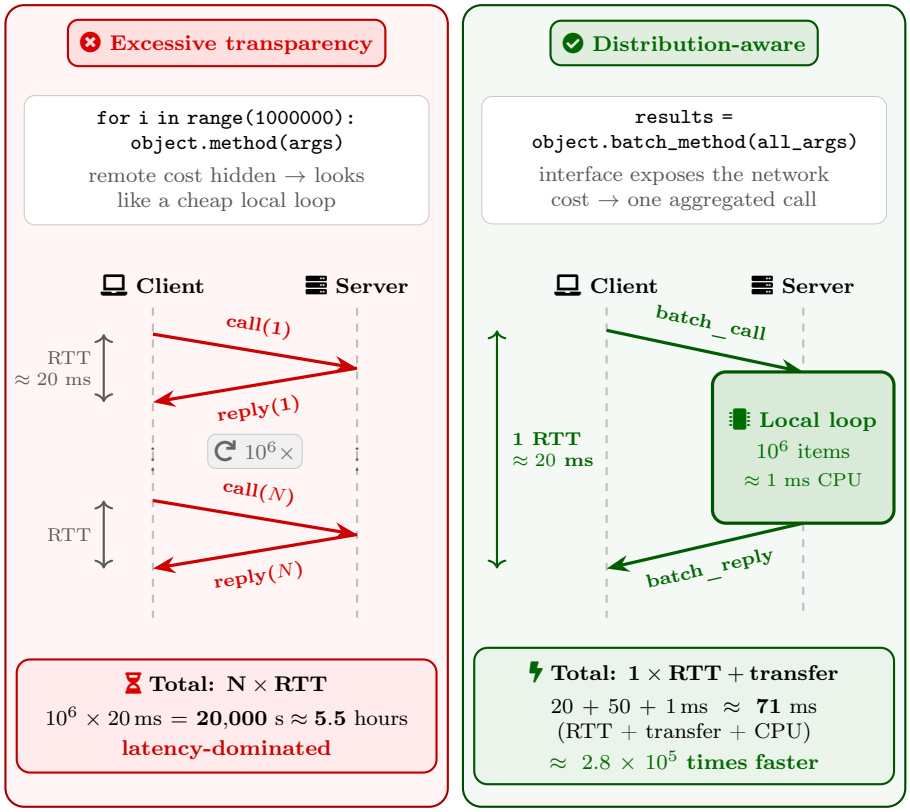


Figure 9: **The latency trap** (illustrative numbers). Excessive transparency can hide network latency and encourage inefficient fine-grained remote calls; exposing distribution costs enables batching and better performance.

Key Takeaways 8

Transparency is about hiding selected aspects of distribution from users/programmers. The main forms answer one simple question: **what exactly are we hiding?**

- **Access**: how the resource is accessed and represented.
- **Location**: where the resource is located.
- **Migration**: that the resource may moved.
- **Relocation**: that it may move while being used.
- **Replication**: that multiple copies exist.
- **Concurrency**: that several users/processes share it.
- **Failure**: that it may fail and recover.
- **Persistence**: whether it is in memory or persistent storage.

And mainly, more transparency is not always better because some aspects cannot really be hidden (especially **communication delay**) and hiding too much can cause bad performance decisions.

Transparency makes a distributed system appear simpler by hiding selected distribution details, but only when hiding them is useful and not misleading.

2 Modeling

After discussing *what a distributed system is*, we now move on to **how we can structure, describe, and reason about one** (i.e., how we can model it). We divide this section into four major parts:

- **Software architecture**: *what infrastructure supports applications?*
- **Run-time architecture**: *how are components organized and connected?*
- **Interaction model**: *what assumptions can we make about time and communication?*
- **Failure model**: *what can go wrong, and what can we guarantee despite failures?*

2.1 Software Architecture and Middleware

Before examining specific distributed architectures such as client-server, peer-to-peer, or REST, we need to understand a more fundamental question: ***how can applications running on different computers communicate and cooperate without having to manage every detail of the underlying platforms?***

Consider two machines:

- Machine A runs Linux and an application written in Python.
- Machine B runs Windows and an application written in Java.

They have different operating systems, programming environments, and potentially different hardware architectures. Nevertheless, we want their applications to communicate and cooperate as part of a single distributed system. There are **two main approaches** to organizing the software infrastructure supporting this interaction: **Network Operating Systems (NOS)** and **Middleware**. Let's examine them in more detail.

Network OS-Based Architecture

A **Network Operating System (NOS)** is an **operating system** that provides facilities for computers to communicate over a network. In a network OS-based architecture, applications rely directly on networking services provided by their respective operating systems.

For example, a programmer may use TCP sockets to exchange messages between two machines. The operating system provides basic facilities such as:

- Establishing network connections.
- Sending and receiving bytes.
- Managing transport protocols such as TCP and UDP.

⚠ However, these facilities do not automatically define how the applications should interpret the exchanged data or coordinate their behavior.

Let's imagine another scenario. *Machine A* wants to invoke an operation on *Machine B*:

```
result = calculate_price(product_id)
```

If `calculate_price()` is implemented on *Machine B*, the programmer cannot simply invoke it as an ordinary local function. Instead, the application must implement the required communication logic:

1. Establish a connection with *Machine B*.
2. Serialize the request into a transferable representation.
3. Send the request across the network.

4. Wait for and receive a response.
5. Deserialize the response.
6. Handle network errors, timeouts, and other failures.

Moreover, the implementations may need adaptations for different programming environments and platform-specific interfaces.

Notice that some of these problems are common to almost every distributed application. ***Why should every application developer have to implement them again?*** This motivates the second approach: **middleware-based architecture**.

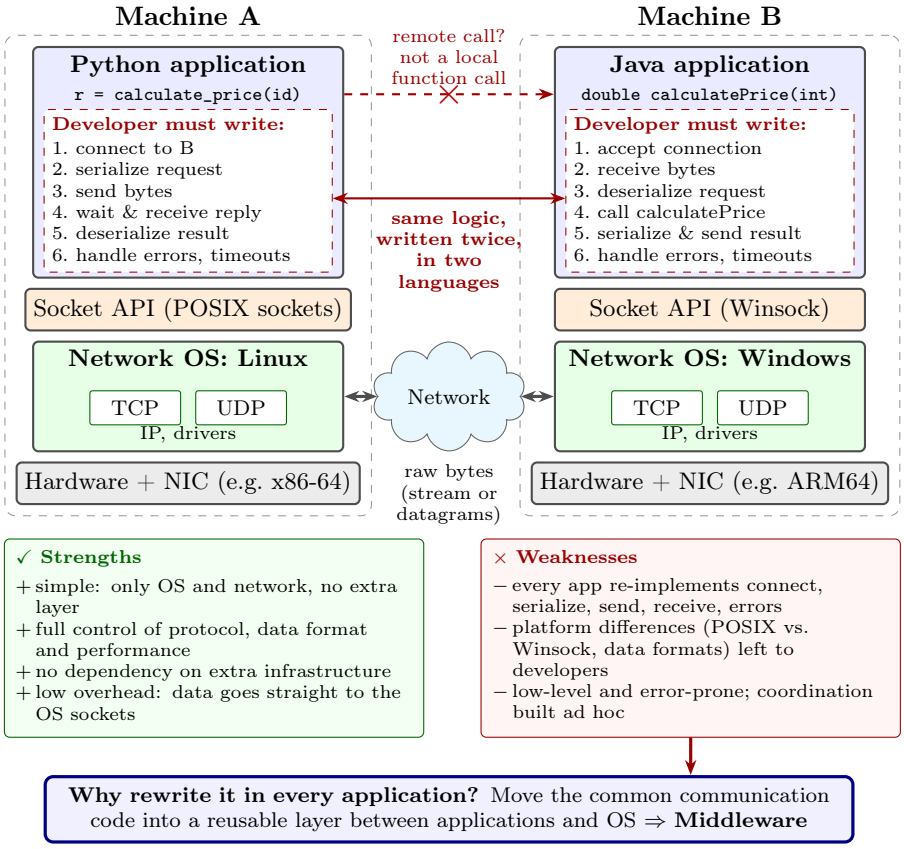


Figure 10: **Network OS-based architecture.** Each application relies directly on the networking facilities (TCP/UDP sockets) of its own operating system. The OS only moves bytes: connecting, serializing, sending, receiving, deserializing and handling failures must be implemented by each application, on each platform. This gives full control, but the same generic logic is rewritten again and again, which motivates middleware.

Middleware-Based Architecture

Middleware is a **software layer** positioned logically between distributed applications and the underlying operating-system/network facilities. Its purpose is to provide reusable services and abstractions that simplify the development of distributed applications.

The central idea is that applications communicate through a **common, well-defined API** instead of dealing directly with all the underlying networking details.

For example, consider a middleware supporting Remote Procedure Calls (RPC). An application may invoke:

```
result = remote_service.calculate_price(product_id)
```

The middleware can take responsibility for: (1) serializing arguments, (2) transmitting requests, (3) receiving responses, and (4) reconstructing results. The programmer works with a higher-level abstraction, like invoking a remote operation rather than manually transmitting bytes over a socket.

The middleware also **hides many differences between platforms** by providing compatible interfaces and communication protocols. Note: the operating systems are still involved. Middleware does not replace their networking facilities, **it builds higher-level abstractions on top of them**.

Aspect	Network OS-Based	Middleware-Based
Communication abstraction	OS networking facilities	High-level APIs
Platform differences	Largely handled by application developers	Partly hidden by middleware
Coordination services	Often developed by applications	Can be provided as reusable services
Development complexity	More low-level concerns	More infrastructure functionality reused
Examples	Direct socket programming	RPC frameworks, message brokers

Middleware simplifies many aspects of distribution, but it does not make remote communication identical to local execution. Network latency, partial failures, and inconsistent system states remain relevant.

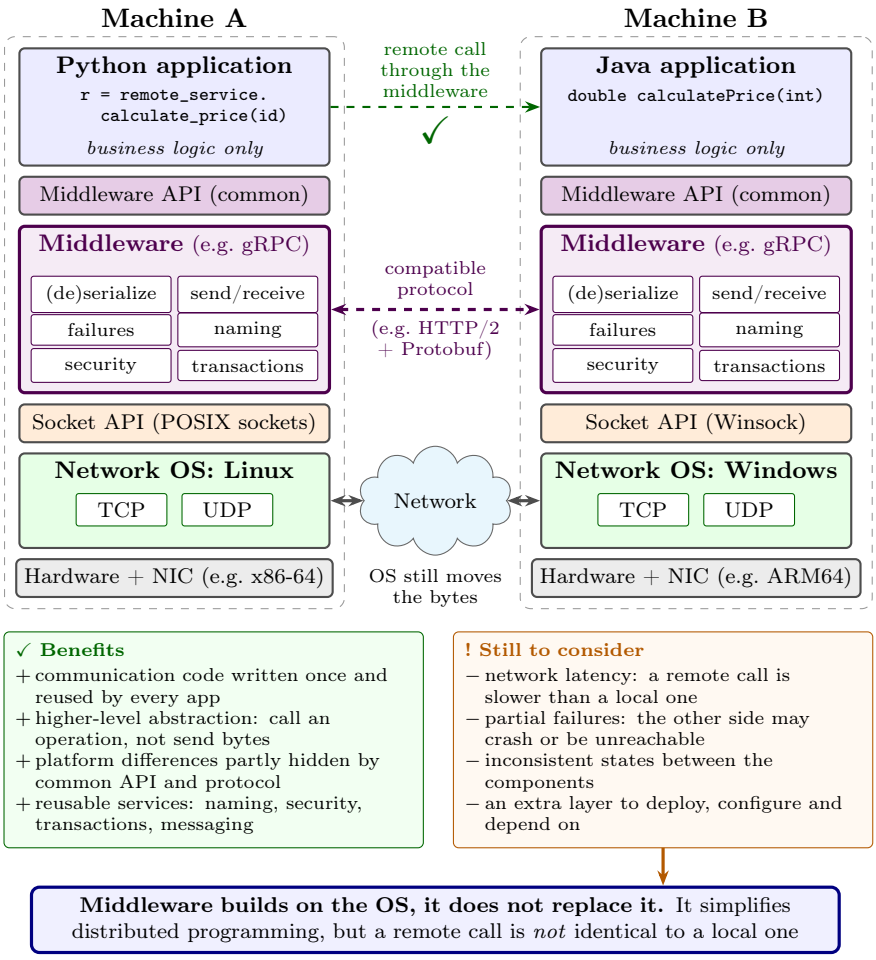


Figure 11: **Middleware-based architecture.** A middleware layer sits between each application and its operating system. The applications use a common, high-level API (e.g. a remote procedure call), while the two middleware components take care of serialization, transmission, failures and other reusable services, talking to each other through a compatible protocol. The operating systems still move the bytes over the network.

Middleware as a Functional Abstraction

We have described **where middleware fits** in a distributed system, and now we need to understand **what services it provides**.

A fundamental property is that middleware typically provides **business-unaware services**. This means that middleware does not need to understand the specific business logic of an application. Consider an e-commerce platform:

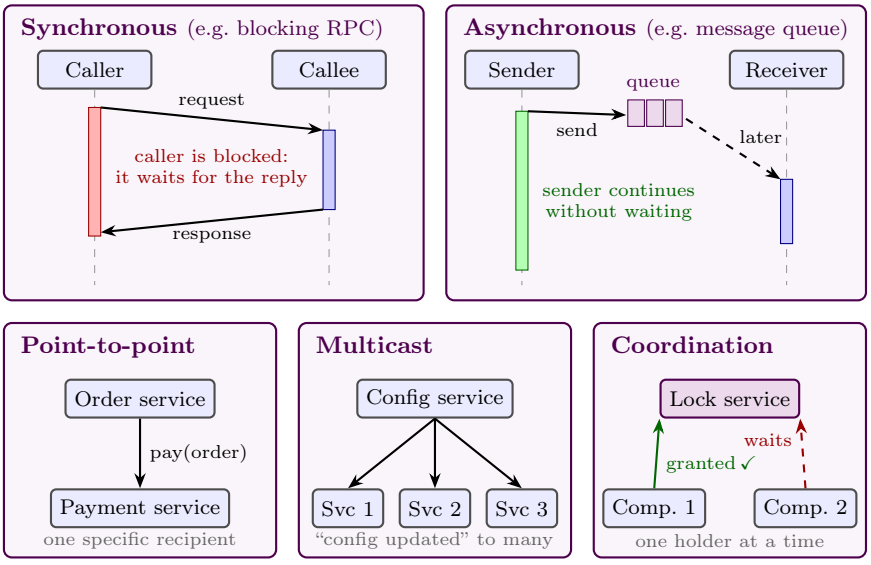
- The application knows what a customer, product, order, or payment represents.
- The middleware does not need to know these concepts.

Instead, it provides generic mechanisms for communicating, coordinating, and managing the distributed components implementing that business logic. We can **classify middleware functionality** into three main categories:

1. **Communication and Coordination Services.** These services determine how distributed components exchange information and coordinate their activities. Communication can be:
   - **Synchronous**, the **caller waits** for a response before continuing.
   - **Asynchronous**, the **sender** can **continue** without waiting for a response.

   Also, communication can be:
   - **Point-to-point**, where a message is sent to a **specific recipient**.
   - **Multicast**, where a message is sent to **multiple recipients** simultaneously.

   In addition to communication, middleware can also provide **coordination primitives**, such as distributed locks, synchronization mechanisms, or coordination protocols.



2. **Special Application Services.** These are reusable, higher-level functionalities provided by middleware to solve common application problems.

Example 1: E-commerce Middleware Services

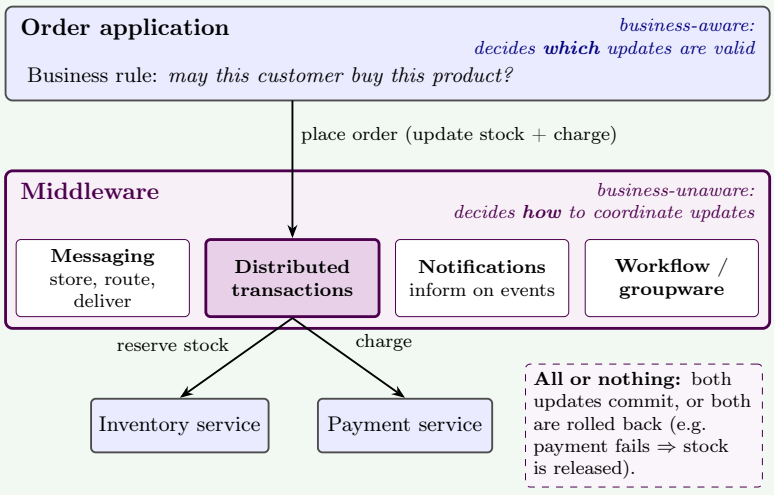
For example, in an e-commerce application, the middleware might provide:

- **Distributed transaction management**: coordinate operations across several components so that they respect transaction guarantees.
- **Messaging services**: store, route, and deliver messages between distributed components.
- **Notification services**: inform interested components when relevant events occur.
- **Workflow and groupware services**: support coordinated activities involving multiple participants or applications.

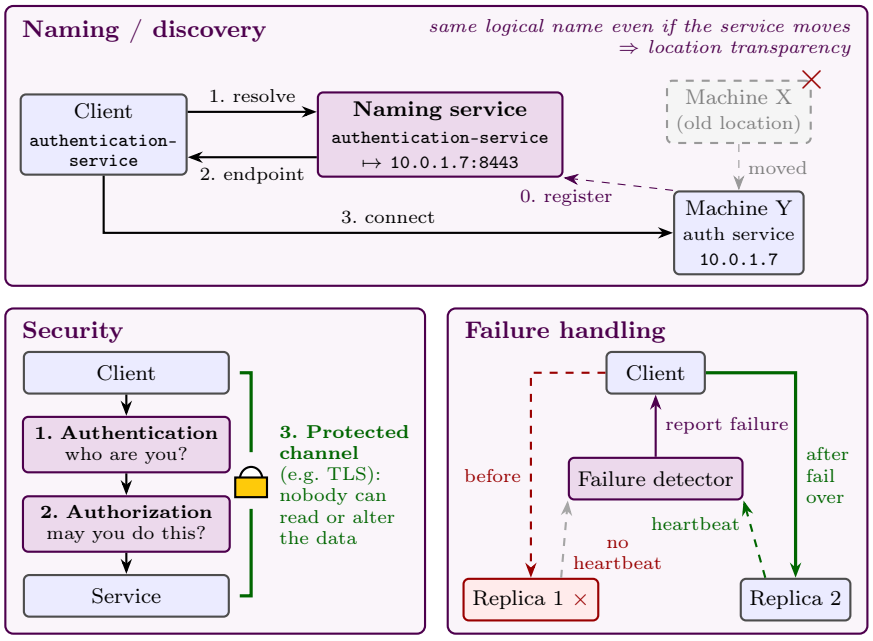
For instance, consider an order that requires updating an inventory service and a payment service. The middleware might provide distributed transaction mechanisms to coordinate the updates. However, it does not determine whether a customer is allowed to purchase a particular product. That decision belongs to the application. The distinction is:

$$\underbrace{\text{How to coordinate updates}}_{\text{Middleware}} \neq \underbrace{\text{Which updates are valid}}_{\text{Application logic}}$$

So **middleware** provides the **mechanisms** (e.g., message queues, transaction coordination), while the **application** decides **how to use them** (e.g., send an order confirmation after payment is successful).



3. **Management Services.** These services address the **operational complexity** of distributed systems. Three fundamental examples are:
   - **Naming**: locating and identifying distributed resources and services.
   - **Security**: providing mechanism for authentication, authorization, and protected communication.
   - **Failure handling**: detecting or reporting failures and supporting recovery mechanisms.



Example 2: Naming and Discovery

For example, suppose a client needs to communicate with an **authentication service**. Instead of hardcoding the IP address of a particular server, it might use a logical service name: `authentication-service`.

A naming or discovery mechanism resolves that name to an appropriate endpoint. If the service moves to another machine, the client may continue using the same logical name. This connects directly to the concept of **location transparency** (page 47).

Middleware can **help achieve transparency**, although how much distribution can actually be hidden depends on its design and on the guarantees offered by the system.

Example 3

Some examples of middleware:

- **gRPC**: remote procedure calls.
- **MPI**: message passing between processes, especially in parallel computing.
- **Apache Kafka**: distributed event streaming and messaging.
- **CORBA**: distributed object interaction.

These technologies solve different problems, but they share the principle of providing reusable abstractions above lower-level networking facilities.

MPI, for example, is a specialized form of communication middleware. It offers operations such as `MPI_Send`, `MPI_Recv`, and collective communication primitives instead of requiring developers to build their own network communication protocols.

Note: middleware does not necessarily need to be a separate server. It might be implemented as libraries inside application processes, as external infrastructure such as message brokers, or as a combination of both.

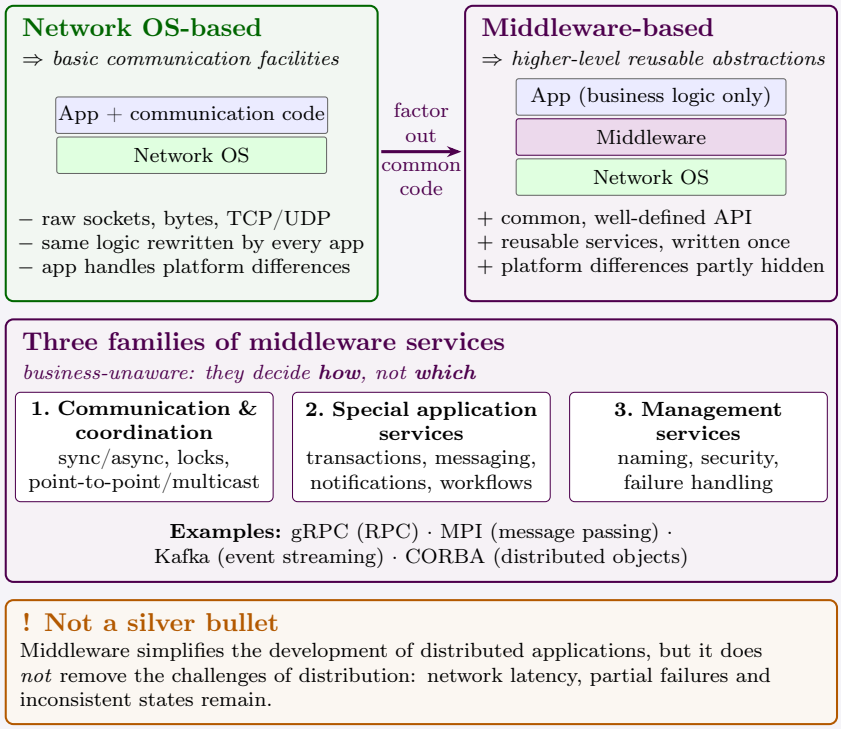
Key Takeaways 1

Network Operating Systems provide basic communication facilities (i.e., moving bytes), while middleware provides higher-level reusable abstractions (i.e., moving objects, messages, or events).

Middleware provides three broad families of services:

1. **Communication and Coordination**: exchanging messages and coordinating components (e.g., RPC, message queues, distributed locks).
2. **Special application services**: transactions, messaging, notifications, workflows.
3. **Management services**: naming, security, failure handling.

Its main **advantage** is reducing the complexity of developing distributed applications by providing common, platform-independent abstractions and reusable services.
In general, middleware simplifies the development of distributed applications, but does **not eliminate the fundamental challenges of distribution**.



In summary, a network OS offers only basic communication facilities, while middleware factors the common logic into higher-level reusable abstractions. Its services fall into three business-unaware families. Middleware simplifies distribution without eliminating its challenges.

2.2 Client-Server and Multi-Tier Architectures

In the previous section, we discussed the **software infrastructure** that enables distributed applications to communicate, particularly Network Operating Systems and middleware. Now we address a different question: once we have the infrastructure to communicate, ***how should we organize the components of a distributed system, and how should they interact?*** This is the purpose of **run-time architecture**.

Run-time Architecture and Architectural Styles

The **Run-Time Architecture** of a distributed system describes how its components are organized and interact while the system is executing. In particular, it identifies three fundamental elements:

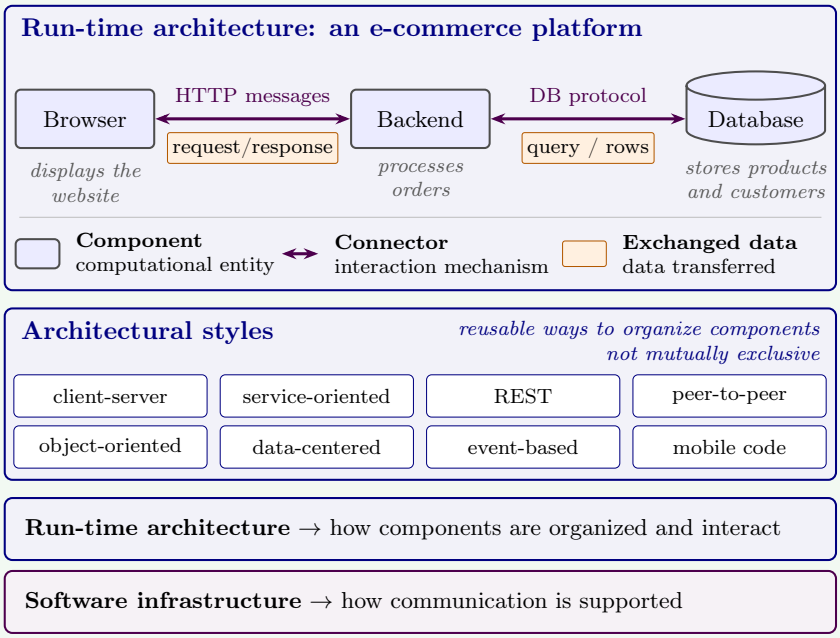
1. **Components**: the computational **entities that perform operations**, such as clients, servers, services, and databases.
2. **Connectors**: the **mechanisms** through which components **interact**, such as message passing, RPC, or shared repositories.
3. **Exchanged data**: the **information transferred** between components, such as requests, responses, events, or data objects.

Instead, an **Architectural Style** is a reusable way of organizing distributed components and defining how they interact.

Rather than designing every distributed system from scratch, we can adopt well-established architectural patterns. Some examples include *client-server*, *service-oriented*, *REST*, *peer-to-peer*, *object-oriented*, *data-centered*, *event-based*, and *mobile-code architectures*. These styles are **not** necessarily mutually exclusive. For example, a system may use client-server interactions while exposing RESTful interfaces and communicating asynchronously through events.

- **Software infrastructure** describes how communication is supported.
- **Run-time architecture** describes how components are organized.

> **Example 4: Run-time Architecture**
>
> For example, consider an e-commerce platform where a browser displays the website, a backend processes orders, and a database stores products and customer information. The run-time architecture describes their roles, relationships, and communication mechanisms.



The figure shows the run-time architecture of an e-commerce platform. While the system executes, *components* (browser, backend, database) interact through *connectors* (HTTP messages, a database protocol) and exchange *data* (requests, responses, queries, rows).

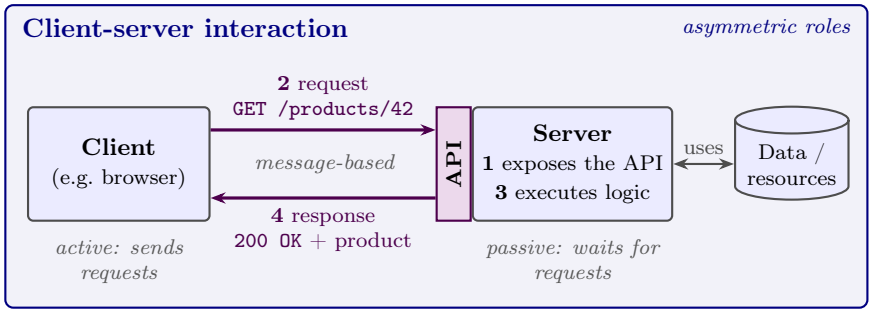
Client-Server Architecture

Client-Server Architecture is one of the most common architectural styles in distributed systems. Its fundamental characteristic is the **asymmetry of component roles**:

- A **server** provides services through a well-defined interface (API).
- A **client** requests and consumes these services.

The typical interaction works as follows:

1. The server exposes operations through an API and *passively* waits for requests.
2. The client sends a request to invoke an operation.
3. The server receives the request, executes the corresponding logic, and possibly accesses other resources.
4. The server sends a response back to the client.



Why is the server described as passive? Because, in the basic client-server model, the server waits for requests instead of spontaneously initiating the interaction with its clients. This does not mean it performs no background processing or cannot contact other services. It describes its **role** in the client-server interaction. For example, an HTTP server waits for incoming HTTP requests, processes them, and produces responses.

Example 5: Client-Server Interaction

Suppose we open an e-commerce website. Our browser sends:

```
1 GET /products/42 HTTP/1.1
2 Host: www.example.com
```

The server processes the request and returns an HTTP response containing the requested resource representation. But the browser does not need to know how the server obtains product information. It only needs to understand the exposed communication interface (API). This is an important architectural principle: service consumers and service provides have a **clear separation of responsibilities**.

Strengths and Limitations. Client-server offers a **clear separation of roles**, **centralized management** of services and data, and **well-defined interfaces**. However, centralization may also introduce:

- **Performance bottlenecks**: a server may become overloaded when too many clients issue requests.
- **Single points of failure**: if a required server fails, its service may become unavailable.
- **Scalability constraints**: additional users may require more server capacity.

Fortunately, these limitations can be mitigated through **replication**, **load balancing**, **caching**, and other techniques.

Finally, *client* and *server* are **roles, not machines**. A single component can act as a server for one client and as a client for another server. This brings us to the concept of **multi-tier architectures**, which are extensions of the client-server model.

Separation of Application Responsibilities

Before discussing tiers, we need to distinguish rapidly three fundamental **classes of functionality** within an application:

- **Presentation**: the *user interface* and *interaction logic*, which is responsible for displaying information and capturing user input.
- **Business Logic**: the *application services*, which process requests, enforce business rules, and coordinate operations.
- **Storage**: the *data services*, which store, retrieve, and manage persistent data.

Example 6: Application Responsibilities

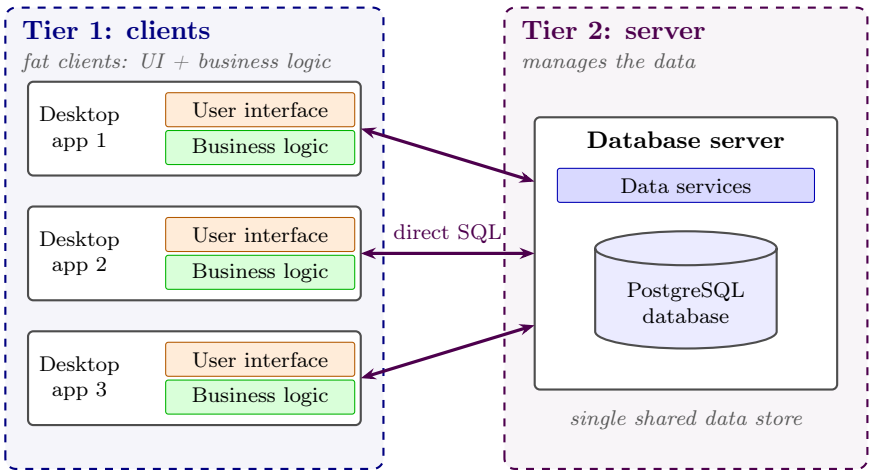
For example, consider an online banking application.

- The user interface displays the account balance and transfer form.
- The applications services check whether a transfer is permitted and process the transaction.
- The data services store account information and transaction history in a database.

These are **logical responsibilities**, they do not necessarily need to execute on separate machines. How we distribute these responsibilities determines the **tiered architecture** of the system.

Two-Tier Architecture

A **Two-Tier Architecture** separates an application into **two communicating execution tiers**. The most traditional arrangement is a client application interacting directly with a database server.



The client handles the UI and much of the business logic, while the server manages the data. The client communicate directly with the database server. Such clients are called **Fat Clients** or **Thick Clients**, because they **contain significant application logic**.

> **Example 7: Two-Tier Architecture**
>
> Imagine a desktop inventory application installed on every employee's computer. The application displays products, calculates discounts, and validates orders locally, while connecting directly to a central PostgreSQL database. This is a typical two-tier architecture.

Note: two-tier does not necessarily imply that business logic resides **entirely** on the client. The defining characteristic is the existence of **two execution tiers**, not one fixed allocation of responsibilities.

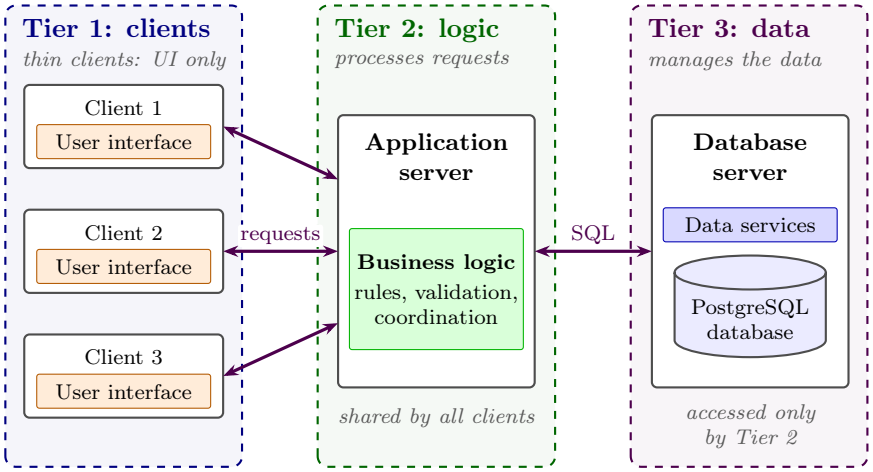
Limitations of Two-Tier Architectures. Two-tier architectures can be simple and effective, but traditional fat-client systems introduce challenges:

- Updating business rules may require **updating each client installation**.
- Clients may need direct access to the database, increasing **security and management complexity**.
- A large number of clients directly accessing one database may cause **scalability problems**.

One solution is to introduce an intermediate tier that centralizes the application logic, leading to a three-tier architecture.

Three-Tier Architecture

A **Three-Tier Architecture** separates **presentation**, **application logic**, and **data management** into three execution tiers.



Example 8: Three-Tier Architecture

Consider the same inventory application of the previous example. Instead of performing all business logic on the user's computer, we introduce an application server that **centralizes the business logic**. Then, the interaction becomes:

1. The client requests information through the user interface.
2. The application server receives the request and executes business logic.
3. If necessary, it queries the database server.
4. The database returns the requested data.
5. The application server processes the result and returns a response to the client.

Notice an important detail: **the *application server* has two different roles**.

1. From the **browser's perspective**, it is a **server** that provides application services.
2. From the **database's perspective**, it is a **client** that requests data.

Therefore, client-server roles are relative to a particular interaction. A component can be a client in one interaction and a server in another.

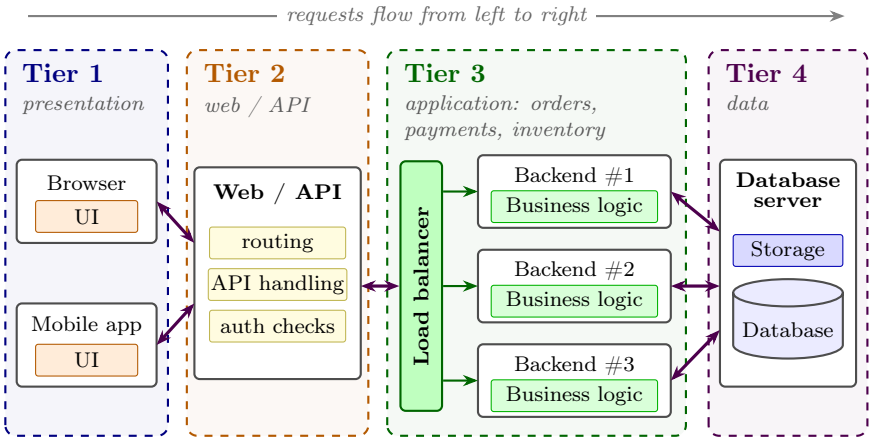
Why introduce an additional tier? Now a natural question arises: *why introduce additional components and communication overhead?* There are four primary advantages:

- ✔ **Centralized business logic.** Rules can be updated on the application server without redeploying the entire client application.
- ✔ **Independent scaling.** If application processing becomes the bottleneck, additional application-server instances can be deployed without necessarily adding database instances.
- ✔ **Security and encapsulation.** Clients no longer need direct access to the database. Data access can be controlled through the application server.
- ✔ **Maintainability.** Presentation, application logic, and data management can evolve more independently when their interfaces are properly designed.

✖ However, there is a **trade-off**: introducing more tiers also introduces **additional network communication**, **deployment complexity**, and **potential failures**.

From Three-Tier to N-Tier Architecture

A **three-tier** architecture may **still be insufficient** for large or complex distributed applications. For example, one application server could become responsible for authentication, payments, inventory, recommendations, and reporting. We might want to decompose this infrastructure into additional tiers. An **N-Tier Architecture** extends the idea of distributing application responsibilities across multiple execution tiers.

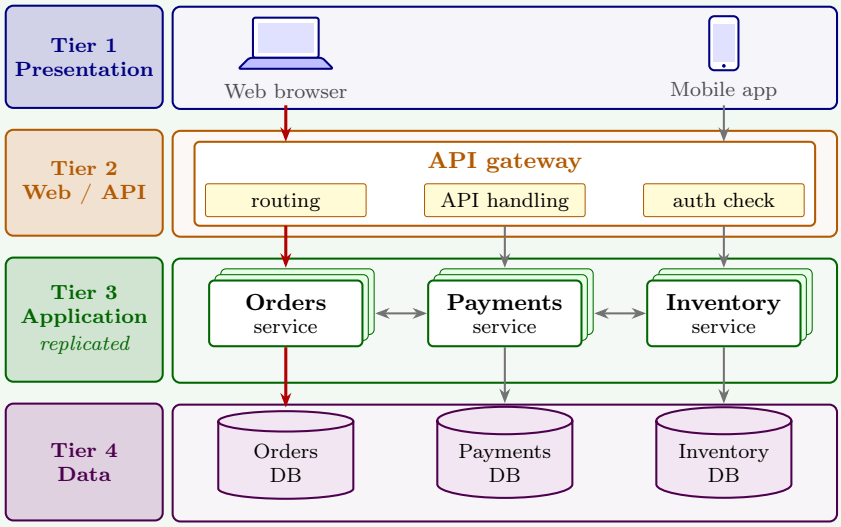


4 tiers ≠ 4 servers: a tier is a logical layer of responsibility; Tier 3 alone runs on several replicated backend instances

Figure 12: N-tier architecture with four tiers: presentation (web and mobile clients), a web/API tier (routing, API handling, auth checks), an application tier (orders, payments, inventory) replicated behind a load balancer, and the data tier. The number of tiers is not the number of servers.

Example 9: N-Tier Architecture

Consider a simplified online store:



one request (`POST /orders`) crosses all four tiers: every tier boundary is a remote call, which adds latency and failure points

This example contains four execution tiers.

- The **presentation tier** contains the user interface and interaction logic, which is responsible for displaying information and capturing user input.
- The **web/API tier** handles incoming requests and may perform routing, authentication checks, or other cross-cutting operations.
- The **application tier** implements domain-specific behavior, such as processing orders, payments, and inventory operations.
- The **data tier** manages persistent data, such as databases and storage systems.

The main idea is to decompose the system into smaller responsibilities that can be deployed, managed, and scaled more **independently**. This can improve modularity, maintainability, scalability, and operational flexibility. However, adding tiers is not automatically beneficial. More tiers also mean more remote calls, latency, failure points, and coordination requirements.

Tiers vs. Components. A **tier** is **not** necessarily a **single server or machine**. For example, the *application tier* might contain five replicated backend instances behind a load balancer. So:

$$\text{Number of Tiers} \neq \text{Number of Servers}$$

⚖ Comparing the Architectures

Architecture	Typical organization	Main advantage	Main limitation
Client-server	Consumers request services from providers	Simple, well-defined roles	Potential server bottlenecks
Two-tier	Client + backend/data server	Simpler deployment structure	Tight coupling and scalability issues in traditional designs
Three-tier	Presentation + application + data	Separation of concerns and independent scaling	Additional communication and deployment complexity
N-tier	Multiple execution tiers	Greater decomposition and flexibility	More complex communication, management, and failure handling

Table 1: Comparison of client-server and multi-tier architectures.

Client-server and multi-tier are **not** competing architectural styles:

- **Client-server** defines the **roles** components play in an **interaction**.
- **Multi-tier** describes **how application** functionality **is distributed** across execution tiers.

A three-tier or N-tier architecture can therefore be constructed from several interconnected client-server relationships.

🖧 Distributed Systems Implications

The architecture of a distributed system has important implications beyond software organization. As soon as different tiers run on different machines, interactions across tiers become distributed interactions. Consider a client requesting a product price from an application server, which queries a database. A lot of complications arise:

- **Latency**: each network request adds communication overhead.
- **Partial failures**: the application server may be operational while the database is unreachable.
- **Concurrency**: multiple application instances may attempt to update the same records.
- **Consistency**: distributed application components must coordinate how shared state is read and modified.

- **Scalability**: increasing the number of application instances does not automatically eliminate a database bottleneck.

Choosing an architecture therefore requires balancing modularity, performance, reliability, scalability, and development complexity.

Key Takeaways 2

The five key takeaways from this section are:

1. **Run-time architecture** defines the components, connectors, and exchanged data of a distributed system.
2. **Client-server** assigns asymmetric roles: clients request services, servers provide them through defined interfaces.
3. Applications commonly separate three responsibilities: presentation, business logic, and data management.
4. **Two-tier, three-tier, and N-tier architectures** describe how those responsibilities are distributed across execution tiers.
5. More tiers can improve separation of concerns and scalability, but increase communication and operational complexity.

The most important conceptual relationship is that **client-server** defines the **interaction roles** of components, while **multi-tier** defines functional deployment organization.

2.3 Service-Oriented Architecture (SOA)

In the previous section, we discussed **client-server and multi-tier architectures**, where components are organized according to their roles and responsibilities. Now we consider a **more complex application**, such as an e-commerce platform. It might need to perform several operations:

- Authenticate customers.
- Check product availability.
- Process payments.
- Arrange shipments.
- Send notifications.

One possibility is to implement all these functionalities within a *single application server*. But ***what if we want each functionality to be developed, deployed, maintained, and potentially reused independently?*** This is the motivation behind **Service-Oriented Architecture (SOA)**.

What is Service-Oriented Architecture?

Service-Oriented Architecture (SOA) is an architectural style[2] in which a distributed application is organized as a **collection of loosely coupled services** that **communicate through well-defined interfaces**.

A **Service** is a self-contained unit of functionality that exposes operations to other components without requiring them to understand its internal implementation.

Example 10: Payment Service

For example, a payment service might expose:

```
PaymentService
    - authorizePayment(amount, currency, cardInfo)
    - capturePayment(transactionId)
    - refundPayment(transactionId)
```

Other applications can **invoke** these operations **without knowing**:

- Which programming language implements the service.
- Which operating system hosts it.
- How its internal algorithms work.
- How it stores its data.

In general, external applications only need to understand the service's **public contract**, which specifies the available operations and how to invoke them. This

[2] An architectural style (page 62) is a reusable way of organizing distributed components and defining how they interact.

is an application of *information hiding*, a principle that we will encounter again when discussing *distributed objects*.

What "*loosely coupled*" means in SOA definition? A fundamental **objective** of SOA is to minimize dependencies between services and their consumers. For example, if a payment service is used by three different applications (e.g., an e-commerce platform, a mobile app, and a subscription management system), we want to ensure that internal changes in the payment service do not break the applications that use it. Therefore, the service's public contract should remain stable, even if its internal implementation evolves over time:

$$\text{SOA} = \text{Independent Services} + \text{Well-defined contracts} + \text{Loose Coupling}$$

However, *loose coupling* does not mean **no coupling at all**. Services or consumers still depend on the service's public contract, availability and behavior. If the service is unavailable or its behavior changes unexpectedly, the consumers may fail to function correctly.

In summary, "***loose coupling***" in SOA means that services have **minimal dependencies on each other**, so one service can change **independently** without requiring changes to its consumers. It does **not** mean zero dependencies, as services still rely on each other's public contracts and availability.

Fundamental Components of SOA

A traditional SOA model identifies 4 fundamental concepts.

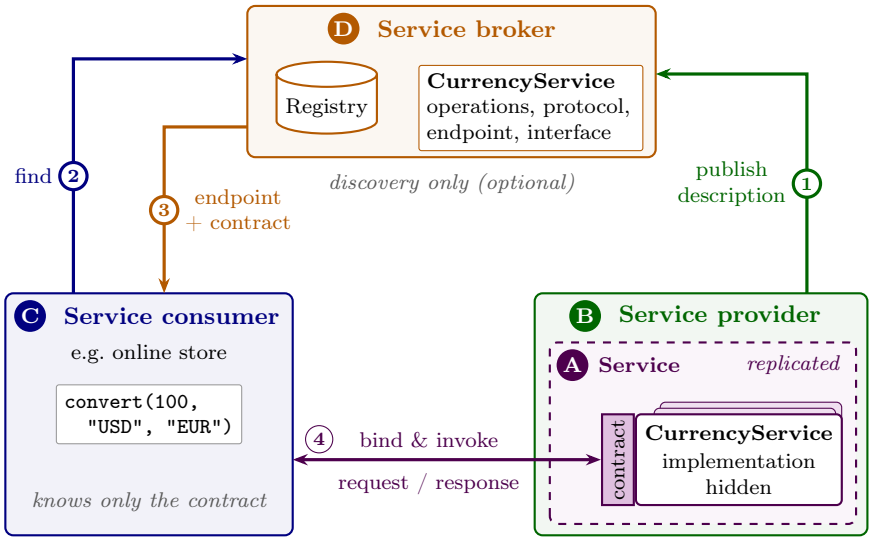


Figure 13: **Fundamental components of SOA.** The provider (B) hosts a service (A) and publishes its description to the broker (D) in step 1. The consumer (C) finds the service in the registry (2), receives its endpoint and contract (3), and then binds to and invokes the service directly at the provider (4).

A. **Service** represents a **reusable unit of functionality** (e.g., authentication, payment, inventory, shipping services). Ideally, each service encapsulates its own implementation and exposes a well-defined interface. A service is a **logical concept**, not necessarily one process or one physical server. For example, a service can be implemented by multiple servers for scalability and fault tolerance.

B. **Service Provider** is the component responsible for **implementing** and **exposing** a service. Its responsibilities include:

   (a) Implementing the service functionality.

   (b) Defining its public interface (contract).

   (c) Making the service accessible through a communication endpoint.

   (d) Optionally publishing its description to a service broker.

> **Example 11: Currency Conversion Service Provider**
>
> For example, a company might operate a currency-conversion service accessible through a network API. The provider handles the actual conversion, while consumers only need to know how to request it.

C. **Service Consumer** is any component that **requires functionality** exposed by another service.

> **Example 12: Currency Conversion Service Consumer**
>
> For example, an online store might invoke:
>
> ```
> price_eur = currency_service.convert(
>     amount=100,
>     source="USD",
>     target="EUR"
> )
> ```
>
> This is conceptual pseudocode, as the actual invocation syntax depends on the communication technology used (e.g., REST, gRPC, SOAP). The consumer needs **to know the service's contract** (i.e., the API), but **not how** the exchange rate is **calculated internally**.

Importantly, a component may simultaneously be a provider and a consumer. For example, an order service might consume a payment service to process payments, while also providing its own order management service to other components. This is similar to the client-server roles, where a server can also act as a client to another server.

D. **Service Broker** provides a mechanism for discovering available services. It maintains a registry containing descriptions of services published by providers.

> **Example 13: Currency Conversion Service Discovery**
>
> Imagine that a consumer needs a service capable of converting currencies. Instead of already knowing the provider's endpoint, it queries the registry to discover a suitable service, and the broker may return information such as the service's identity, its supported operations, its communication protocol, its network endpoint, a description of its interface. Then, the consumer can use this information to communicate with the provider.

The broker typically does not execute the service's business logic. Its main responsibility is **service discovery**. Also, a broker is **not mandatory for every SOA implementation**: services can be configured or discovered through other mechanisms.

⇄ Publish-Find-Bind-Invoke Model

The interaction among providers, consumers and brokers follows a characteristic sequence of steps, known as the **Publish-Find-Bind-Invoke Model**.

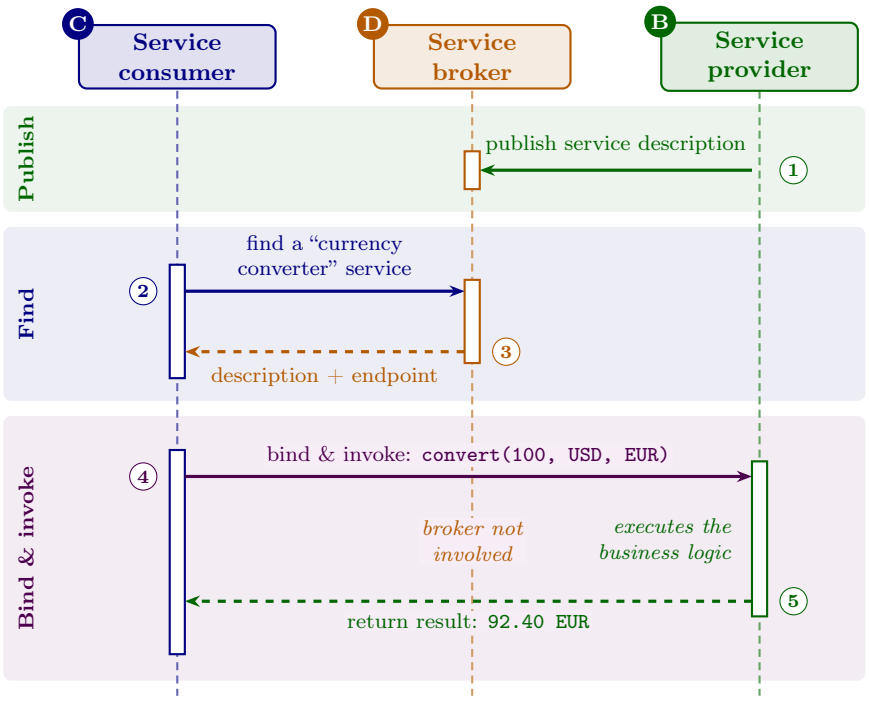


Figure 14: **The publish-find-bind-invoke model.** The provider publishes its service description to the broker (1). The consumer finds a suitable service (2) and receives its description and endpoint (3). It then binds to the provider and invokes the operation directly (4), and the provider returns the result (5); the broker is not involved in the invocation.

Step 1: Publish. The **provider** registers information describing its service.

> **Example 14: Currency Conversion Service Description**
>
> For example, a currency conversion service might publish:
>
> ```
> Service: CurrencyConversion
> Operation: convert(amount, source, target)
> Endpoint: https://currency.example.com/service
> ```

Step 2: Find. The **consumer** requests information about a service satisfying certain requirements.

> **Example 15: Currency Conversion Service Find Request**
>
> For example, find a service that converts amounts between currencies.

Step 3: Discover. The **broker** returns the description of a suitable service, including the information needed to contact it.

Step 4: Bind and Invoke. The **consumer** establishes the necessary communication relationship (**binding**) and invokes the operation through the service's interface.

Binding does not necessarily mean opening a persistent network connection. It means associating the consumer with the selected service and the protocol details required to invoke it.

Step 5: Response. The **provider** executes the requested operation and returns the result.

Why is this useful? Suppose the original currency-conversion provider becomes **unavailable** or is **replaced** by a different implementation.

- ✔ A **discovery mechanism** can help consumers find an alternative provider exposing a compatible service contract.
- ✔ This supports **flexibility** and **reduces dependence on specific implementations**.
- ✘ However, SOA doesn't automatically provide **fault tolerance** or **guarantee** that an **equivalent provider exists**.

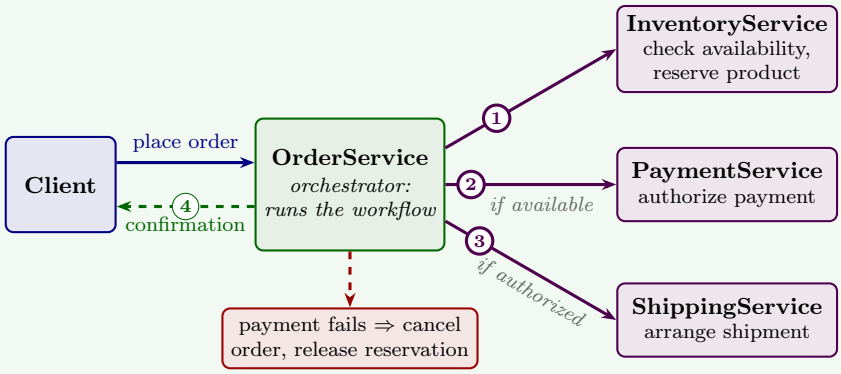
Service discovery can contribute to **location transparency** (page 47), a consumer can identify a service logically without embedding the physical location of its implementation. SOA also promotes **interoperability** (page 36) and **modularity** by exposing standardized service contracts.

Service Orchestration

Until now, we have considered a consumer invoking an individual service. But real distributed applications often need to combine multiple services to accomplish a single objective. This introduces the concept of **Service Orchestration**. **Orchestration** is the process of coordinating the invocation of multiple services according to a workflow to achieve a particular goal.

Example 16: Processing an Online Order

Consider an Order Service that must coordinate three other services to process an online order.



When a customer places an order, the orchestrator `OrderService` might execute the following workflow:

1. Invoke `InventoryService` to check product availability.
2. If the product is available, invoke `PaymentService` to authorize the payment.
3. If authorization succeeds, reserve the product and arrange shipment through the appropriate services (e.g., `ShippingService`).
4. Return the order confirmation.

If the payment fails, the orchestrator must decide what happens next, such as cancelling the order or releasing a reservation.

The important point is that each service implements a specific functionality, while the orchestrator **coordinates** how those functionalities contribute to a larger operation.

Traditional Web Services

We have discussed SOA as an architectural style, independently of the technology used to implement it. Now we examine **one of its historically important implementations**: **SOAP-based Web Services**.

In general, a **Web Service** is a software system that enables interoperable machine-to-machine communication over a network. Traditional Web Services typically **rely on three standards**:

- **Web Services Description Language (WSDL)** specifies the **service contract**, like the operations it exposes, the messages it can send and receive, and the communication details (e.g., protocols, endpoints).

  Specifically, WSDL is an XML-based language used to **describe the public interface** of a Web Service. It defines information such as:

  - Which operations the service provides.
  - Which messages and data types are accepted.
  - Which messages are returned.
  - Which protocol and message format are used.
  - Where the service can be accessed.

  So in general, WSDL tells the consumer **what the service exposes** and **how to invoke it**.

  Example 17: Calculator Service Contract

  Consider a simple calculator Web Service. Its interface might conceptually define:

  ```
  CalculatorService

  Operation:
      add(a: integer, b: integer) -> integer

  Input:
      a, b

  Output:
      result
  ```

  A **WSDL document** describes this contract in a **standardized**, **machine-readable** way. The standardized description allows different systems, like a Java client and a C# service, to communicate without needing to know each other's internal implementations. The consumer only needs to understand the interface and communication conventions. Furthermore, development tools can even generate client-side proxy code from a WSDL description.

- **Simple Object Access Protocol (SOAP)** defines an **XML-based messaging protocol** for exchanging requests and responses.

  In general, WSDL describes the interface, but **we still need to exchange actual invocation messages**. This is the role of **SOAP**, which is an XML-based messaging protocol defining **how structured messages are exchanged between communicating components**.

  Example 18: SOAP Request and Response

  For example, a **consumer** might invoke the calculator's `add` operation by **sending an illustrative SOAP request**:

```xml
<soap:Envelope
    xmlns:soap="http://www.w3.org/2003/05/soap-envelope">

    <soap:Body>
        <add xmlns="http://example.com/calculator">
            <a>5</a>
            <b>3</b>
        </add>
    </soap:Body>

</soap:Envelope>
```

  The **service executes** the operation and **returns a corresponding SOAP response**:

```xml
<soap:Envelope
    xmlns:soap="http://www.w3.org/2003/05/soap-envelope">

    <soap:Body>
        <addResponse xmlns="http://example.com/calculator">
            <result>8</result>
        </addResponse>
    </soap:Body>

</soap:Envelope>
```

  Notice the structure:

  - `Envelope` identifies the message as a SOAP message.
  - `Header` (optional) carries additional information, potentially including security or routing metadata.
  - `Body` contains the application-specific message content.

  Note that SOAP specifies the **messaging structure** and **processing rules**, but it does not determine the application's business logic (i.e., what the service actually does).

- **Universal Description, Discovery, and Integration (UDDI)** defines a **registry framework** for publishing and discovering services.

  Precisely, UDDI is a standardized framework for **publishing** and **discovering Web Services** through registries. It implements the *service broker* concept introduced earlier on page 75.

  In other words, UDDI tells consumers which services are available and where to find information about them. UDDI is historically important for understanding SOA, but many real-world Web Services are accessed through known endpoints or other discovery mechanisms without relying on UDDI registries.

SOA activity	Technology	Responsibility
Describe	WSDL	Defines the service contract
Publish / Discover	UDDI	Registers and discovers services
Bind / Invoke	SOAP	Exchanges invocation messages

Table 2: **Traditional Web Services technologies.**

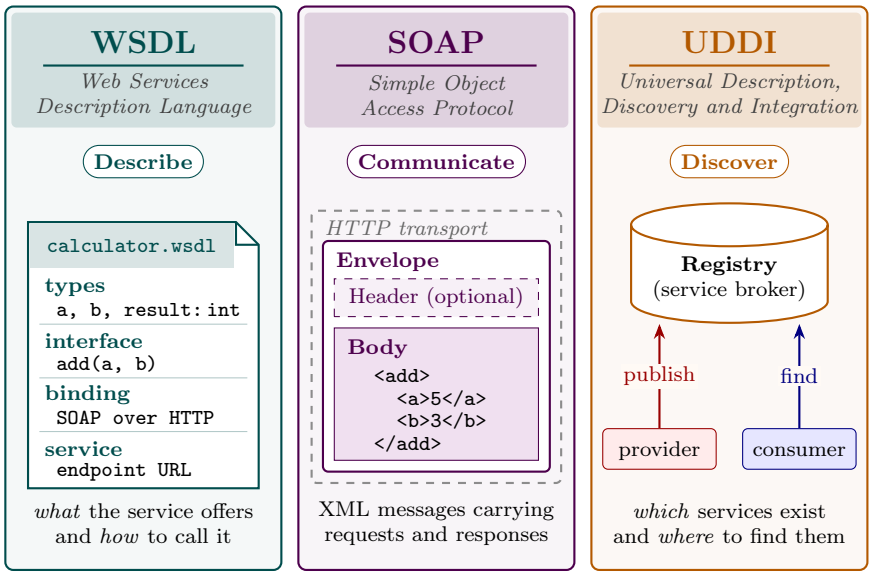


Figure 15: **The three standards of traditional Web Services.** WSDL describes the service contract (types, interface, binding, endpoint), SOAP carries request and response messages (usually over HTTP), and UDDI provides a registry where providers publish and consumers find services.

Example 19: How WSDL, SOAP, and UDDI Work Together

Consider a Java application that needs a remote calculator.

1. The calculator provider makes its service description available through WSDL.
2. The provider may register its service in a UDDI registry.
3. The Java application discovers the calculator service through the registry.
4. It retrieves the WSDL description and configures or generates a client.
5. The client sends a SOAP request to invoke `add(5, 3)`.
6. The provider processes the request and responds with the result `8`.

Note that the broker is involved in discovery, not necessarily in every operation invocation.

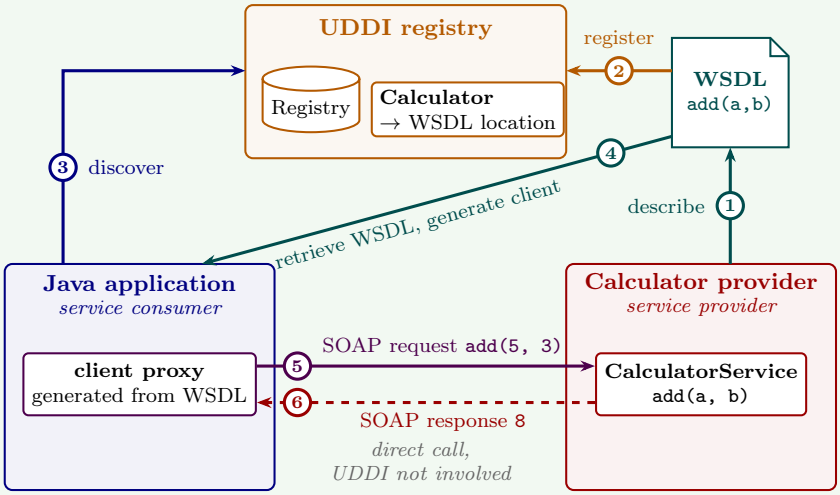


Figure 16: **How WSDL, SOAP and UDDI work together.** The provider describes its calculator service with WSDL (1) and registers it in UDDI (2). The Java application discovers it (3), retrieves the WSDL and generates a client (4), then invokes `add(5, 3)` with a SOAP request (5) and receives `8` in the SOAP response (6). The SOAP exchange goes directly from consumer to provider, without UDDI.

⚖ SOA vs. Client-Server and Multi-Tier Architectures

After introducing SOA and traditional Web Services, a natural question arises: ***isn't SOA simply another name for client-server?*** This question is understandable, as both SOA and client-server architectures involve a consumer (client) invoking a service (server). However, there are important distinctions because **emphasize different aspects of a system**, even though they can coexist.

Architecture	Main question
Client-Server	*Who requests functionality, and who provides it?*
Multi-Tier	*How are application responsibilities distributed across execution tiers?*
SOA	*How can a system be composed of loosely coupled, reusable services?*

For example, consider the classic e-commerce example:

- It uses **client-server** interactions because consumers (i.e., customers) invoke providers (i.e., servers) to request and receive services.
- It can be **three-tier** because presentation (i.e., web browser), application logic (i.e., order processing), and data management (i.e., database) are separated into different tiers.
- It follows **SOA principles** because authentication, payment, inventory, and shipping are exposed as independently defined services.
- It might use **SOAP-based Web Services** as the communication technology between some components (e.g., between the order processing service and the payment service).

So, SOA is not a replacement for client-server or multi-tier architectures. Also, SOA is not synonymous with SOAP, because SOAP-based Web Services are **just one way to implement SOA**. The broader SOA principles can also be implemented with other communication technologies.

> **Key Takeaways 3**
>
> 1. **SOA organizes distributed applications around services.** Services are reusable, loosely coupled units of functionality exposed through well-defined interfaces.
> 2. **Four fundamental concepts define the traditional model.**
>   - **Service**: the functionality being offered.
>   - **Provider**: implements and exposes the service.

- **Consumer**: discovers and invokes the service.
- **Broker**: maintains information used for service discovery.

3. **The typical SOA interaction follows a publish-find-bind-invoke model.**
4. **Orchestration combines services into workflows.** An orchestrator coordinates multiple service invocations to accomplish a larger objective.
5. **Remember the three traditional Web Services technologies.**

$$\text{WSDL} \rightarrow \text{Describe}$$
$$\text{SOAP} \rightarrow \text{Communicate}$$
$$\text{UDDI} \rightarrow \text{Discover}$$

In general, SOA changes the way we think about building distributed applications. Instead of thinking only about distributing application components across machines, we think about composing independently defined services into larger applications. This makes the service contract, loose coupling, discovery, and composition central architectural concerns.

References

[1] Margara Alessandro Cugola Gianpaolo Saverio. Distributed systems. Slides from the HPC-E master's degree course on Politecnico di Milano, 2026.

Index

A
Architectural Style 62

C
Client-Server Architecture 63

D
Distributed System 23–25

E
Eight Fallacies of Distributed Computing 19

F
Fat Clients 66

M
Middleware 28, 55

N
N-Tier Architecture 68
Network Operating System (NOS) 53

P
Pervasive Computing 8
Publish-Find-Bind-Invoke Model 76

R
Run-Time Architecture 62

S
Service 72, 74
Service Broker 75
Service Consumer 75
Service Orchestration 78
Service Provider 74
Service-Oriented Architecture (SOA) 72
Simple Object Access Protocol (SOAP) 80
SOAP-based Web Services 79

T
Thick Clients 66
Three-Tier Architecture 67
Two-Tier Architecture 66

U
Ubiquitous Computing 8
Universal Description, Discovery, and Integration (UDDI) 81

W
Web Service 79
Web Services Description Language (WSDL) 79